



Gymnázium J. K. Tyla

Úvod do agentického AI a design autonomní pipeline pro softwarový vývoj

Návrh a realizace systému DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

KONCEPT

Anotace

Práce se zabývá návrhem a realizací autonomního systému pro vývoj softwaru, který přebírá rutinní kroky vývojového procesu — od přijetí požadavku přes jeho interpretaci a naplánování až po vytvoření a ověření změny. Teoretická část shrnuje principy kontinuální integrace, řízení verzí a orchestrace jazykových modelů. V praktické části je popsán systém DarkFactory, jeho architektura a governance, a jsou vyhodnoceny výsledky jeho nasazení na reálných repozitářích.

Klíčová slova

autonomní systémy, softwarové inženýrství, kontinuální integrace, orchestrace agentů, jazykové modely

Annotation

This thesis deals with the design and implementation of an autonomous software engineering system that takes over the routine steps of the development process — from intake of a request through its interpretation and planning to the creation and verification of a change. The theoretical part summarises the principles of continuous integration, version control and the orchestration of language models. The practical part describes the DarkFactory system, its architecture and governance, and evaluates the results of its deployment on real repositories.

Keywords

autonomous systems, software engineering, continuous integration, agent orchestration, language models

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	3
Annotation	3
Keywords	3
1 Úvod	7
1.1 Motivace	7
1.2 Cíl práce	7
1.3 Metodika	8
2 Teoretická část	9
2.1 Řízení verzí	9
2.1.1 Větvě a jejich role	9
2.1.2 Model pull requestu	9
2.2 Kontinuální integrace	10
2.2.1 Požadované kontroly	10
2.2.2 Sestavení a artefakty	10
2.3 Jazykové modely a jejich orchestrace	10
2.3.1 Transformer a LLM	10
2.3.2 Tokeny, tokenizér a embedding	11
2.3.3 Mechanismus pozornosti a jeho typy	13
2.3.4 Multimodální modely	17
2.3.5 Context	18
2.3.6 Prompt a kontextové inženýrství	19
2.3.7 Agent vs Chatbot	21
2.3.8 Harness a System Prompt	21
2.3.9 ReAct smyčka	21
2.3.10 Vytváření nástrojů	22
2.3.11 Dovednosti, skripty a zachytané body	23
2.3.12 MCP Server	24
2.3.13 Meta Harness	24
2.3.14 Subagenti	24
2.3.15 Workflows neboli Graph Engineering	25
2.4 Human in the loop neboli člověk ve smyčce	26
2.4.1 Schvalovací body	26

2.4.2	Dohledatelnost	26
2.4.3	Eskalace a transparence selhání	26
3	Praktická část	27
3.1	Cíl a rozsah systému	27
3.2	Architektura	27
3.2.1	Sdílení místo kopírování	28
3.2.2	Jediný konfigurační soubor	28
3.3	Životní cyklus požadavku	30
3.4	Popis prostředí repozitáře	31
3.4.1	Domény a prostředí	31
3.4.2	Deklarace jako doplněk rozpoznávání	32
3.5	Orchestrace napříč poskytovateli	33
3.6	Ověřování změn	34
3.6.1	Optimalizace běhu a cachování závislostí	34
3.6.2	Automatické hlášení a deduplikace selhání	34
3.7	Automaticky generovaná dokumentace	35
3.8	Systém revizních značek pro lidský dohled nad textem	36
4	Výsledky a diskuse	38
4.1	Profil nasazených repozitářů a testovací prostředí	38
4.2	Sjednocení pracovních postupů	38
4.3	Rozšíření na texty	40
4.4	Provozní metriky a spolehlivost agentních běhů	40
4.5	Chyby, které se projeví až v provozu	42
4.6	Diskuse	43
4.7	Omezení	44
5	Závěr	46
	Seznam příloh	49
A	Obsah přiloženého média	50
B	Schéma konfiguračního manifestu darkfactory.json	51
C	Sdílené workflow pro GitHub Actions	53
D	Systémové prompty plánovacího a kódovacího agenta	55
E	Protokol revizních značek v sazebním systému Typst	57
E.1	Vizuální reprezentace jednotlivých prvků v sazbě	57
	Seznam zdrojů	59

Seznam obrázků a tabulek

Obrázek 1	Geometrická reprezentace sémantických vztahů v embeddingovém prostoru: A) Klasická vektorová aritmetika pojmů ($\vec{v}(\text{král}) - \vec{v}(\text{muž}) + \vec{v}(\text{žena}) \approx \vec{v}(\text{královna})$), B) Izomorfní algebraické relace v programování (funkce $\rightarrow$ metoda, proměnná $\rightarrow$ atribut).....	12
Tabulka 1	Empirické porovnání tokenové náročnosti ekvivalentního větného významu v angličtině a češtině (tokenizér <code>cl100k_base</code>).....	13
Tabulka 2	Srovnání architektur organizace hlav mechanismu pozornosti a jejich dopad na paměťovou náročnost KV cache.	16
Obrázek 2	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	22
Obrázek 3	Dekompozice komponent systému DarkFactory: třívrstvá architektura propojující platformu GitHub, výkonné skriptové jádro a izolované běhové prostředí s modely.	27
Obrázek 4	Stavový diagram životního cyklu požadavku v systému DarkFactory: přechody mezi stavy od založení issue přes schvalovací brány (Human Gates), tvorbu větve a PR až po automatické sloučení a uzavření úkolu.	30
Tabulka 3	Rozpoznávání prostředí podle souboru popisujícího balíček.	31
Obrázek 5	Procesní schéma generování a striktní validace živé dokumentace v CI pipeline.	36
Tabulka 4	Kvantitativní vyhodnocení autonomních agentních běhů napříč sledovanými repozitáři.	41

1 Úvod

Vývoj softwaru se za posledních dvacet let z velké části zautomatizoval. Sestavení programu, spuštění testů, kontrola stylu i nasazení do provozu dnes obstarávají stroje a nikdo je nepovažuje za práci hodnou lidského času. Jeden krok však zůstal ruční: samotná změna zdrojového kódu. Právě tam se tvoří většina prodlev — mezi okamžikem, kdy někdo popíše požadavek, a okamžikem, kdy se změna dostane k uživatelům.

Výrobní průmysl zná pojem *dark factory*, tedy „temná továrna“: provoz, který běží bez lidské obsluhy, a proto v něm nemusí svítit. Nejde o představu úplného vyloučení člověka — i temná továrna má konstruktéry, kteří rozhodují, co se bude vyrábět — nýbrž o vyloučení člověka z opakujících se úkonů. Tato práce zkoumá, jak stejný princip uplatnit ve vývoji softwaru.

1.1 Motivace

S rozšířením velkých jazykových modelů se objevila možnost automatizovat i psaní kódu. Většina dostupných nástrojů však řeší jen dílčí krok: vygenerují návrh změny, který někdo musí zasadit do procesu, ověřit a schválit. Chybí popis toho, jak má takový nástroj zapadnout do vývojového procesu jako celku — kdo schvaluje, co se stane při selhání a jak se pozná, že je výsledek správný.

Právě tato mezera je předmětem práce. Zajímavá není otázka, zda model dokáže napsat kód; to je dnes doloženo. Zajímavá je otázka, jaké okolí musí kolem takového modelu vzniknout, aby jeho výstupu bylo možné důvěřovat.

1.2 Cíl práce

Cílem této práce je navrhnout, realizovat a ověřit systém, který automatizuje vývojový proces od přijetí požadavku po vytvoření ověřené změny, aniž by se vzdal lidského schválení v rozhodujících bodech.

Dílčí cíle:

1. Popsat současný stav automatizace vývoje softwaru a orchestrace jazykových modelů.
2. Navrhnout architekturu systému, který provede požadavek celým procesem.
3. Systém realizovat a nasadit na reálné repozitáře.
4. Vyhodnotit jeho chování a pojmenovat omezení, na která v provozu narazil.

V návaznosti na stanovené cíle si práce klade tři konkrétní výzkumné otázky:

- **VO1:** Do jaké míry lze rutinní fáze softwarového vývoje (analýza, implementace, testování, dokumentace) automatizovat pomocí orchestrace jazykových modelů při zachování deterministických záruk a nulové regrese v hlavní větvi?
- **VO2:** Jaké jsou fundamentální limitující faktory současných LLM při samostatné práci nad reálným repozitářem z hlediska kapacity kontextového okna, degradace pozornosti a stability nástrojů?

- **VO3:** Umožňuje zavedení jednotného deklarativního manifestu (`darkfactory.json`) a centralizovaného CI workflow škálovat autonomní pipeline napříč heterogenními projekty s minimálními dodatečnými náklady na údržbu?

 **Návrh na vylepšení:** Operacionalizace výzkumných otázek: Doporučuji v úvodu stručně uvést kvantitativní metriky a kritéria úspěšnosti pro zodpovězení VO1–VO3 (např. cílová míra průchodnosti CI bez regrese, práh akceptovatelné latence a definice konstantní údržbové složitosti $O(1)$), aby bylo v kapitolách 4 a 5 zřejmé, na základě jakých exaktních prahů byly otázky vyhodnoceny.

1.3 Metodika

Práce je z povahy tématu konstrukční a inženýrská: primárním výstupem je funkční, plně integrovaný systém a empirické vyhodnocení jeho provozní spolehlivosti v reálném vývojovém prostředí. Postup odpovídá iterativnímu inženýrskému cyklu: po analýze teoretických východisek následoval návrh modulární architektury, implementace řídicího metaharnesu a jeho postupné nasazení na tři typově odlišné repozitáře:

1. **DarkFactory:** mateřský repozitář systému (Python, GitHub Actions, metaharnes).
2. **OdbornaPrace-paper:** repozitář samotného rukopisu této práce (doména textu a sazby v systému Typst).
3. **ChessWithQuests:** aplikační projekt s herní logikou.

Empirické ověření probíhalo longitudinálním sledováním reálných integračních běhů v prostředí GitHub Actions nad skutečnými požadavky (GitHub Issues) a pull requesty. Místo syntetických laboratorních benchmarků (např. izolovaného vyhodnocování na datasetech typu SWE-bench) se výzkum soustředil na end-to-end spolehlivost v produkčních podmínkách: sledována byla schopnost pipeline projít celým životním cyklem bez uvážnutí, četnost vyčerpání kontextu či API limitů, chování záchranných mechanismů při rotaci modelů a zejména kvalitativní a kvantitativní analýza chyb, které se projeví v reálném provozu. Získané poznatky sloužily k průběžné optimalizaci a zpevnění mantinelů celého systému.

2 Teoretická část

Tato kapitola vymezuje pojmy, o které se opírá praktická část: řízení verzí, kontinuální integraci, orchestraci jazykových modelů a princip zapojení člověka do smyčky (*Human-in-the-loop*). Cílem není vyčerpávající přehled, nýbrž zavedení pojmů v podobě, v jaké s nimi pracuje navržený systém.

2.1 Řízení verzí

Systém pro řízení verzí uchovává historii změn zdrojového kódu. Distribuovaný model, jehož nejrozšířenějším zástupcem je Git, se od centralizovaného liší tím, že každý vývojář má úplnou kopii historie (1). Změny lze proto vytvářet a zkoumat i bez spojení se serverem a slučovat je až ve chvíli, kdy jsou hotové.

2.1.1 Větev a jejich role

V moderních distribuovaných systémech správy verzí (zejména v systému Git (1)) je repositář formálně modelován jako orientovaný acyklický graf (*Directed Acyclic Graph*, DAG), jehož uzly tvoří neměnné objekty revizí (*commits*) provázané kryptografickými hashy (SHA-1 či SHA-256) na své rodičovské stavy. Větev v tomto modelu nepředstavuje fyzickou kopii souborů, nýbrž odlehčený, pohyblivý ukazatel (*ref*) na konkrétní uzel grafu.

Tato grafová architektura má klíčový význam pro bezpečné zapojení autonomních agentů do vývojového procesu:

- **Izolace stavu:** Agent operuje ve vyhrazené větvi (např. `feature/...` nebo `agent/...`), která odbočuje z hlavní vývojové linie (`main`). Veškeré pokusné mutace souborového systému, mezistavy a ladící kroky zůstávají striktně izolované, aniž by ohrožily stabilitu produkčního kódu nebo práci ostatních členů týmu.
- **Deterministický audit:** Každý krok agenta lze reprezentovat jako atomický commit s přesným časovým otiskem, autorskými metadaty a odkazem na kontextové zadání. Vzniká tak neměnná a zpětně ověřitelná historie změn.
- **Strategie slučování:** Při integraci hotové větve do hlavní linie se uplatňují různé topologické strategie: přímý posun ukazatele (*fast-forward*), vytvoření explicitního slučovacího uzlu (*merge commit*), nebo sloučení celé sekvence dílčích mezikroků agenta do jediného čistého uzlu (*squash and merge*). Právě squashování je v autonomních pipeline preferováno, neboť eliminuje šum v podobě neúspěšných pokusů modelu a v hlavní větvi zanechává pouze finální, ověřený přírůstek.

2.1.2 Model pull requestu

Sloučení větve do hlavní vývojové linie se v moderním kolaborativním softwarovém inženýrství odehrává prostřednictvím modelu **pull requestu** (PR, na platformě GitLab též

Merge Request). Jde o formalizovaný procesní uzel, v němž autor větve předkládá navržený diff kódu k revizi dříve, než dojde k jeho trvalému začlenění do chráněné hlavní větve. V kontextu autonomního vývoje plní pull request dvě nezastupitelné funkce:

1. **Strojová validační brána:** Na vytvoření nebo aktualizaci PR reaguje integrační server (CI), který v izolovaném kontejneru spustí sadu automatizovaných testů, typových kontrol a bezpečnostních linterů. Tím je objektivně ověřeno, že syntetický kód generovaný modelem splňuje stanovené standardy a nezpůsobuje regresi stávající funkcionality.
2. **Lidská schvalovací brána (Human Gate):** Pull request poskytuje přehledné rozhraní zobrazující řádkový diff změn, výsledky automatických testů a strukturovaný popis úprav. Člověk v roli recenzenta (*code reviewer*) tak může provést finální sémantickou kontrolu a rozhodnout o schválení či zamítnutí změny. Pull request tím představuje ideální architektonický styčný bod pro princip člověka ve smyčce (*Human-in-the-loop*).

2.2 Kontinuální integrace

Kontinuální integrace (angl. *continuous integration*) je praxe, při níž se každá změna automaticky sestaví a otestuje (2). Namísto dlouhých období, kdy se změny hromadí a slučují až na konci, se ověřuje průběžně a v malých dávkách, takže chyba je odhalena blízko svému vzniku.

2.2.1 Požadované kontroly

Ke kontinuální integraci patří pojem *požadovaných kontrol* (angl. *required checks*): množina úloh, které musí skončit úspěšně, jinak nelze změnu sloučit. Tím se z kvality stává vlastnost vynucovaná strojem, nikoli pouze dohodou mezi vývojáři.

Návrh požadovaných kontrol má jedno nezřejmé úskalí. Úloha, která se za jistých okolností „přeskočí“, nehlásí žádný výsledek; je-li přitom uvedena mezi požadovanými, zablokuje slučování napořád. Kontrola proto musí vždy skončit nějakým závěrem — i kdyby jím bylo konstatování, že v daném repozitáři není co dělat.

2.2.2 Sestavení a artefakty

Výsledkem sestavení bývá *artefakt*: spustitelný soubor, knihovna, nebo — jak ukazuje praktická část — vysázený dokument. Automatizace vydávání verzí spojuje artefakt se značkou v historii, takže ke každé vydané verzi existuje doložitelný výstup.

2.3 Jazykové modely a jejich orchestrace

2.3.1 Transformer a LLM

Transformer je architektura hlubokých neuronových sítí představená společností Google v roce 2017 v rámci výzkumu strojového překladu. Na rozdíl od starších architektur (např. rekurentních sítí RNN), které sekvence zpracovávaly krok po kroku a trpěly ztrátou dlouhodobého kontextu, využívá transformer mechanismus zvaný *attention* (pozornost).

Ten modelu umožňuje paralelně vyhodnocovat sémantické souvislosti mezi všemi tokeny v sekvenci bez ohledu na jejich vzájemnou vzdálenost.

LLM neboli *Large Language Model* (velký jazykový model) je rozsáhlá neuronová síť postavená na architektuře transformeru a předtrénovaná na textových datech o objemu stovek miliard až bilionů tokenů. Díky mechanismu pozornosti model dokáže zachytit hluboké syntaktické i sémantické struktury přirozeného jazyka, což se navenek projevuje schopností generalizace, abstrakce a generování korektního zdrojového kódu.

Tento architektonický přelom popsala publikace *Attention Is All You Need* (3). Původní model byl koncipován jako **Encoder-Decoder** (kodér-dekodér) pro strojový překlad: obousměrný kodér nejprve zkomprimoval celou vstupní větu a dekodér za pomoci křížové pozornosti (*cross-attention*) generoval překlad v cílovém jazyce.

Ve vývoji softwaru a moderních agentních systémech se však naprostým standardem stala architektura **Decoder-only** (např. GPT, Claude, LLaMA či DeepSeek). Tyto modely pracují čistě autoregresivně — predikují vždy následující nejpravděpodobnější token na základě celého předcházejícího kontextu. Instrukce, pravidla, kontext repozitáře i rozepsaný kód tvoří jedinou společnou sekvenci, což umožňuje plynulé doplňování kódu i přímé generování volání nástrojů. Architektura pouze s dekodérem navíc vykazuje vynikající vlastnosti při škálování parametrů a efektivní správě KV cache v dlouhých kontextech.

2.3.2 Tokeny, tokenizér a embedding

Text, se kterým člověk pracuje ve formě slov a vět, není pro neuronovou síť přímo srozumitelný. Model interně operuje pouze s čísly a maticovými operacemi. Aby bylo možné přirozený jazyk zpracovat, musí projít procesem tokenizace a následného převodu na vektorové reprezentace.

Základní jednotkou, kterou model vnímá, je **token**. Token nemusí odpovídat celému slovu ani jednotlivému znaku — moderní jazykové modely využívají tzv. sub-word tokenizaci (nejčastěji algoritmy jako Byte-Pair Encoding či WordPiece). Běžná slova jsou často reprezentována jediným tokenem, zatímco méně častá slova, odborné výrazy nebo slova v jazycích s bohatou morfologií a diakritikou (jako je čeština) jsou rozložena do více dílčích tokenů. Průměrně jeden token v angličtině odpovídá přibližně 3 až 4 znakům. V češtině je spotřeba tokenů na slovo znatelně vyšší, což má přímý dopad na efektivní kapacitu kontextového okna i výpočetní náklady inference.

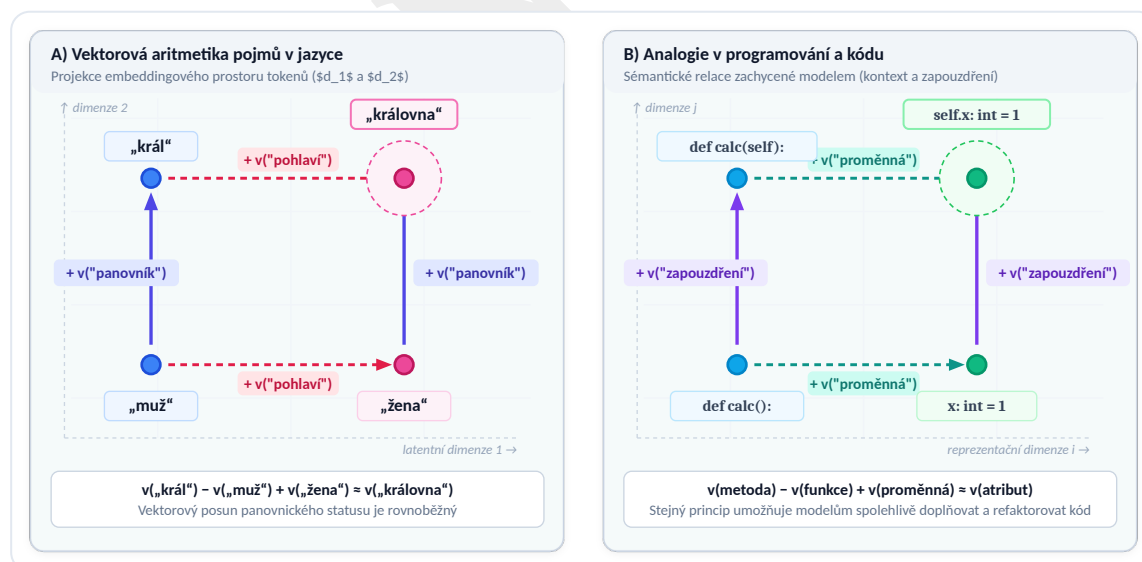
Převod mezi surovým textovým řetězcem a posloupností celočíselných identifikátorů (token IDs) zajišťuje **tokenizér**. Jedná se o deterministický program, který vychází z pevně daného slovníku (tzv. vocabulary, obvykle čítajícího 32 000 až 128 000 unikátních tokenů). Tokenizér provádí obousměrný proces: při vstupu rozseká text na tokeny a přiřadí jim číselné

indexy, při výstupu naopak generované indexy skládá zpět do souvislého textu (tzv. detokenizace).

Samotné číslo tokenu však nenese žádnou informaci o jeho významu. Proto následuje vrstva zvaná **embedding** (vektorové vnoření). Každý token je převeden na spojitý vícerozměrný vektor (typicky o dimenzi 4 096 či 8 192 čísel). V tomto geometrickém prostoru jsou slova s podobným významem či kontextem umístěna blízko u sebe (např. měreno kosinovou podobností).

Jednotlivé geometrické směry a posuny v prostoru navíc odpovídají konkrétním sémantickým relacím a abstraktním vlastnostem. Zjednodušeným a klasickým příkladem fungování tohoto prostoru je vektorová aritmetika pojmů: pokud vezmeme vektor reprezentující pojem „žena“ a přičteme k němu vektor reprezentující posun k „panovnickému stavu / královskému statusu“, výsledný bod v prostoru leží nejbližší reprezentaci slova „královna“ ($\vec{v}(\text{žena}) + \vec{v}(\text{královský status}) \approx \vec{v}(\text{královna})$). Model tak nepracuje se slovy jako s izolovanými symboly, ale manipuluje se vztahy mezi nimi jako s algebraickými posuny ve vícerozměrném prostoru — což se v programování projevuje schopností zachytit vztahy mezi deklarací proměnné, jejím použitím a typovým kontextem.

Aby architektura transformeru zohlednila také pořadí tokenů v sekvenci, přičítá se k sémantickému embeddingu poziční kódování (positional encoding, dnes standardně rotační embedding RoPE). Teprve takto vzniklé vektory vstupují do mechanismu pozornosti k dalšímu výpočtu.



Obrázek 1: Geometrická reprezentace sémantických vztahů v embeddingovém prostoru: A) Klasická vektorová aritmetika pojmů ($\vec{v}(\text{král}) - \vec{v}(\text{muž}) + \vec{v}(\text{žena}) \approx \vec{v}(\text{královna})$), B) Izomorfní algebraické relace v programování (funkce $\rightarrow$ metoda, proměnná $\rightarrow$ atribut).

Grafické znázornění na Obrázek 1 ilustruje, jak vícerozměrné vektorové vnoření zachycuje abstraktní sémantické relace. Na levém panelu (A) je patrné, že vektorový posun reprezen-

tující přechod k panovnickému stavu ($\vec{v}(\text{panovník})$) má téměř identický směr a velikost jak mezi „mužem“ a „králem“, tak mezi „ženou“ a „královnou“. Pravý panel (B) ukazuje analogický princip v programovacím kódu: model vnímá vztah mezi volně stojící funkcí a metodou zapouzdřenou ve třídě jako paralelní posun ve vektorovém prostoru k relaci mezi globální proměnnou a atributem objektu. Díky této prostorové struktuře dokáže LLM provádět konzistentní refaktoring a typovou inferenci.

Praktický dopad sub-word tokenizace na efektivitu a kapacitní limity agenta demonstruje Tabulka 1. Identické větné sdělení vyžaduje v češtině 33 tokenů oproti pouhým 12 tokenům v angličtině (měřeno tokenizérem `cl100k_base`). Český text tak spotřebovává 2,75× více prostoru v kontextovém okně a úměrně tomu zvyšuje finanční náklady i latenci inference. V autonomních vývojových pipeline je proto optimální vést vnitřní systémové prompty, technické plány a strukturované logy v angličtině, zatímco lidská interakce v zadání může probíhat v mateřském jazyce vývojáře.

Jazyk	Znění věty	Znaky	Tokeny
Angličtina	An autonomous software development system analyzes requirements and proposes code changes.	90	12
Čeština	Autonomní systém pro vývoj softwaru analyzuje požadavky a navrhuje změny kódu.	79	33

Tabulka 1: Empirické porovnání tokenové náročnosti ekvivalentního větného významu v angličtině a češtině (tokenizér `cl100k_base`).

2.3.3 Mechanismus pozornosti a jeho typy

Zatímco tokenizér a embedding transformují diskrétní text do spojitého vektorového prostoru, funkčním jádrem architektury transformeru je mechanismus pozornosti (*attention mechanism*). Pozornost umožňuje modelu dynamicky vyhodnocovat sémantické relace mezi jednotlivými tokeny v sekvenci bez ohledu na jejich vzájemnou vzdálenost, a překonat tak fundamentální limit starších rekurentních architektur (RNN a LSTM), které trpěly postupnou ztrátou kontextu.

2.3.3.1 Matematická podstata: Scaled Dot-Product Attention

Základní stavební jednotkou představenou v práci (3) je skalovaný skalární součin (*Scaled Dot-Product Attention*). Pro každý vstupní vektor tokenu se lineární projekcí generuje trojice vektorů:

- **Dotaz (Q – Query):** Vektor reprezentující informaci, kterou aktuální token v kontextu vyhledává.
- **Klíč (K – Key):** Vektor nesoucí charakteristiku a profil tokenu, podle něhož je identifikován.

- **Hodnota (V – Value):** Vektor obsahující vlastní sémantický obsah, který je v případě shody předán do dalších vrstev.

Míra relevance mezi libovolnou dvojicí tokenů je dána skalárním součinem QK^T . Výsledná matice afinity je škálována odmocninou dimenze klíče $\sqrt{d_k}$, což zabraňuje prudkému růstu číselných hodnot a následné saturaci funkce softmax (která by vedla k vymizení gradientů při trénování). Výstupní kontextová reprezentace je definována jako vážený součet hodnot V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Z matematické formulace vyplývá klíčová vlastnost: výpočet součinu QK^T pro sekvenci délky N vyžaduje sestavení matice o rozměrech $N \times N$. Výpočetní složitost i paměťové nároky proto rostou **kvadraticky** $O(N^2)$ vzhledem k délce kontextu, což při zpracování rozsáhlých repozitářů a dlouhých agentních historií představuje primární výkonnostní limit.

2.3.3.2 Směrové a funkční typy pozornosti

Podle původu vektorů Q , K a V a směru toku informací se rozlišují tři základní funkční typy:

1. **Vlastní pozornost (Self-Attention):** Vektory Q , K i V jsou odvozeny ze stejné vstupní sekvence. Každý token v kontextu tak přímo interaguje se všemi ostatními tokeny kódové báze (např. provázání volání funkce s její definicí v jiném modulu).
2. **Kauzální (maskovaná) pozornost (Causal / Masked Attention):** Klíčový princip autoregresivních modelů (*Decoder-only*). Aby model při predikci kódu nemohl „nahlížet do budoucna“, je na matici skalárních součinů před aplikací softmaxu uvalena kauzální maska ($M_{i,j} = -\infty$ pro $j > i$). Tím je matematicky zaručeno, že token na pozici i smí reflektovat výhradně předcházející tokeny na pozicích $j \leq i$.
3. **Křížová pozornost (Cross-Attention):** Dotazy Q pocházejí z generujícího dekodéru, zatímco klíče K a hodnoty V jsou přebírány z externí reprezentace — např. z vizuálního enkodéru zpracovávajícího screenshoty či diagramy v multimodálních architekturách.

2.3.3.3 Architektury organizace hlav (Head Architectures)

V praxi transformer nepočítá pozornost pouze jednou, nýbrž v paralelních projekčních podprostorech zvaných **hlavy** (*Multi-Head Attention* – MHA). Během inference se však ukázalo, že ukládání mezistavů klíčů a hodnot do mezipaměti grafické karty (*KV Cache*) pro všechny hlavy a vrstvy představuje kritické paměťové úzké hrdlo. To vedlo k vývoji několika architektonických variant (srovnání uvádí Tabulka 2):

1. **Multi-Head Attention (MHA):** Původní architektura ((3)). Každá z h hlav má vlastní nezávislé projekční váhy pro Q , K i V . Poskytuje maximální reprezentační kapacitu, avšak velikost KV cache v paměti GPU roste strmě s počtem hlav a délkou kontextu.
2. **Multi-Query Attention (MQA):** Architektura navržená Shazeerem (2019). Všechny dotazové hlavy sdílejí jedinou společnou hlavu klíčů a hodnot (K a V). Velikost KV cache v paměti klesá h -násobně, což radikálně zrychluje generování tokenů a snižuje paměťové nároky, avšak za cenu mírné degradace kvality u úloh vyžadujících komplexní logické uvažování.
3. **Grouped-Query Attention (GQA):** Zlatý standard současných produkčních modelů (např. LLaMA 3 či Mistral; (4)). Dotazové hlavy jsou rozděleny do g skupin, přičemž každá skupina sdílí vlastní pár hlav K a V . Představuje optimální kompromis: dosahuje kvality srovnatelné s MHA při paměťové úspoře blízké MQA (obvykle 12,5–25 % původní velikosti KV cache).
4. **Multi-head Latent Attention (MLA):** Moderní architektura vyvinutá pro modely DeepSeek-V2/V3. Místo prostého sdílení hlav komprimuje vektory klíčů a hodnot do nízkorozměrného latentního prostoru pomocí společné maticové komprese (*low-rank joint compression*). Během inference se do KV cache ukládá pouze kompaktní latentní vektor, což dramaticky redukuje paměťovou stopu a umožňuje obsluhu masivních kontextových oken.

Typ pozornosti	Hlavy Q	Hlavy K, V	Velikost KV Cache	Charakteristika a využití
MHA	h	h	100 % (báze)	Původní Transformer, GPT-3. Maximální exprese, vysoké nároky na VRAM.
MQA	h	1	$\frac{1}{h}$ (3–6 %)	Falcon, PaLM. Extrémní úspora paměti, možný mírný pokles přesnosti.
GQA	h	g ($1 < g < h$)	$\frac{g}{h}$ (12–25 %)	LLaMA 3, Mistral. Špičkový standard kombinující přesnost a propustnost.
MLA	h	Latentní komprese	5–10 %	DeepSeek-V2, DeepSeek-V3. Nízkorozměrná latentní projekce s vysokou věrností.

Tabulka 2: Srovnání architektur organizace hlav mechanismu pozornosti a jejich dopad na paměťovou náročnost KV cache.

2.3.3.4 Hardwarové a algoritmické optimalizace pro škálování kontextu

Kvadratická závislost $O(N^2)$ si vyžádala vývoj algoritmických a nízkoúrovňových hardwarových akcelerací:

- **Sliding Window a Sparse Attention (SWA):** Pozornost je omezena na lokální okno W nejblížešších sousedních tokenů (např. $W = 4096$ v modelu Mistral). Ačkoli v jedné vrstvě model nevidí celou sekvenci, skládáním vrstev nad sebou efektivní kontextové pole roste, zatímco výpočetní složitost klesá na lineární $O(N \times W)$.
- **FlashAttention:** Převratná optimalizace navržená Dao et al. ((5), následovaná architekturami FlashAttention-2 a FlashAttention-3). FlashAttention neprovádí žádnou ztrátovou

aproximaci — matematický výsledek je exaktní. Podstata spočívá v hardwarovém uvědomění (*IO-awareness*): namísto ukládání ohromné mezilehlé matice pozornosti $N \times N$ do pomalé globální paměti grafické karty (HBM) rozděluje výpočet na bloky (tzv. *tiling*) a provádí normalizaci online algoritmem softmaxu přímo v rychlé čipové mezipaměti SRAM procesoru GPU. Tím dramaticky redukuje paměťové přenosy a umožňuje praktické škálování kontextového okna na stovky tisíc až miliony tokenů.

2.3.4 Multimodální modely

Architekturu transformeru lze aplikovat i mimo oblast zpracování přirozeného jazyka. V moderní praxi lze tokenizovat v podstatě jakýkoli diskrétní či spojitý signál — ať už jde o rastrový obraz, sekvenci snímků videa, zvukové vlny nebo trajektorie pohybů v robotice. Zásadní posun nastává ve chvíli, kdy model disponuje oddělenými projekčními vrstvami pro různé typy vstupů (např. kombinací textového tokenizéru a vizuálního enkodéru, jako je ViT — *Vision Transformer*) a mechanismus pozornosti (*cross-attention*) operuje společně nad tokeny textu i obrazu. Model tak dokáže propojovat vizuální a textové sémantické reprezentace ve sdíleném vektorovém prostoru. V kontextu automatizovaného vývoje softwaru a autonomních pipeline (jako je systém DarkFactory) se multimodální schopnosti uplatňují ve třech klíčových oblastech:

1. **Vizuální regresní testování:** Model dokáže porovnat referenční snímek uživatelského rozhraní se stavem vygenerovaným v CI (např. při testování webových komponent bezhlavým prohlížečem) a identifikovat nežádoucí posuny rozvržení či stylové chyby.
2. **Diagnostika chyb z artefaktů CI:** Při selhání integračních testů může pipeline předat agentovi screenshot chybové obrazovky nebo interaktivního prvku, z něhož agent rozpozná příčinu selhání snáze než z pouhého textového stack trace.
3. **Interpretace grafických zadání:** Vývojář může v zadání úkolu (GitHub Issue) specifikovat požadovanou změnu formou náčrtku, wireframu či diagramu komponent. Multimodální agent takový grafický podklad přímo zanalyzuje a převede jej na strukturovaný kód.

⚠ Chyba / Nesrovnalost k opravě: Rozpor mezi teoretickým popisem a stavem implementace: Podkapitola 2.3.3 uvádí využití multimodálních modelů v systému DarkFactory (vizuální regresní testování, diagnostika screenshotů z CI, wireframy) jako hotovou součást pipeline. V praktické části ani ve výsledcích však podpora pro obrazové vstupy integrována není (pipeline pracuje čistě s textovými diffy a logy). Je nutné text přeformulovat a uvést, že jde o teoretický potenciál či plánované rozšíření architektury.

2.3.5 Context

2.3.5.1 Turn

Každému diskrétnímu kroku ve výměně informací mezi okolním prostředím a jazykovým modelem se v agentních architekturách říká tah (*turn*). Na rozdíl od jednoduchého konverzačního rozhraní, kde dochází pouze ke střídání uživatele a asistenta, agentní smyčka (typicky implementující vzor ReAct (6)) rozlišuje tři základní typy tahů:

1. **Tah uživatele či prostředí (*User / Environment Turn*):** Vnáší do kontextu nové zadání, externí událost (např. spuštění GitHub webhooku) nebo doplňující instrukce.
2. **Tah modelu (*Model / Assistant Turn*):** Reprezentuje jedno spuštění inference nové sítě. Model na základě dosavadní historie emituje buď finální textovou odpověď, nebo strukturovaný požadavek na vyvolání nástroje (*tool call*).
3. **Tah vykonání nástroje (*Tool Execution Turn*):** Běhové prostředí (harness) provede požadovanou operaci — např. spuštění skriptu v terminálu či čtení souboru — a její výsledek vloží do kontextu jako syntetický tah určený pro navazující uvažování modelu. Striktní oddělení těchto fází a jejich deterministická serializace do historie kontextu mají zásadní dopad na výpočetní efektivitu: umožňují inferenčnímu serveru plně využívat cachování klíčů a hodnot (*KV cache*), neboť neměnná historie předchozích tahů nemusí být při každém kroku znovu přepočítávána.

2.3.5.2 Context Window

Model je schopen přijmout pouze omezený objem vstupu; tomuto limitu se říká kontextové okno (*Context Window*) a vyjadřuje se v počtu tokenů. Na rozdíl od slovníku (*vocabulary*), který vymezuje pouze repertoár známých tokenů, je maximální délka kontextového okna určena architekturou pozičního kódování (např. škálováním frekvenčních bází v RoPE) a hardwarovou náročností mechanismu pozornosti — kvadratickou složitostí $O(N^2)$ a velikostí alokované KV cache v operační či grafické paměti.

2.3.5.3 Compaction

Představuje proces sumarizace a zkrácení historie, který řídící harness iniciuje ve chvíli, kdy zaplnění kontextového okna dosáhne stanoveného prahu. Zpravidla jde o vyvolání modelu se specifickou systémovou instrukcí pro bezztrátovou syntézu dosavadního průběhu sezení a kompletním protokolem dosavadní komunikace. Výsledný zkrácený kontext následně v kontextovém okně nahradí původní rozsáhlou historii kroků.

2.3.5.4 Context Rot a evaluace vybavování (*Needle In A Haystack*)

 **Hlubková kritika / Oponentura:** Teoretická naivita rekurzivní komprese: Popsaný mechanismus komprese (Compaction) se v textu tváří jako elegantní a bezproblémové řešení, v reálu jde však o destruktivní ztrátovou kompresi. Model při rekurzivním zkracování historie trpí silným

konfirmačním zkreslením — sumarizuje to, co sám považuje za podstatné, čímž nevratně maže přesná čísla řádků, jemné sémantické hrany zadání, negativní mantinely („tohle nikdy neměň“) a detaily chybových hlášení. Práce se vůbec nezabývá fundamentálním fenoménem sémantického posunu (*semantic drift*) po několika kolech komprese, ani moderními bezeztrátovými alternativami (hierarchická RAG paměť, persistentní graf stavu projektu či selective KV cache eviction).

Označuje empiricky zdokumentovanou degradaci schopnosti modelu věnovat rovnoměrnou pozornost všem částem historie (tzv. jev *Lost in the Middle* (7)). Čím plnější je kontextové okno, tím méně jsou modely schopny spolehlivě vybavovat jemné detaily z úvodu sezení a dodržovat negativní omezující podmínky zadání.

Ke standardizovanému testování schopnosti modelu přesně vybavovat informace z rozsáhlého kontextového okna slouží syntetický evaluační test **Needle In A Haystack** (NIAH – „jehla v kupce sena“). Princip spočívá ve vložení izolovaného, na kontextu nezávislého faktu (např. unikátního klíče či specifického nastavení konfigurace) do různé relativní hloubky (od 0 % do 100 %) dlouhého distraktoru textu (čítajícího desítky tisíc až miliony tokenů). Model je následně vyzván k zodpovězení dotazu, jehož vyřešení závisí výhradně na této vložené informaci.

Zatímco moderní frontier modely dosahují na jednoduchém syntetickém testu NIAH téměř stoprocentní úspěšnosti napříč celou délkou okna, v reálném softwarovém vývoji vyvstává zásadní omezení: práce v repozitáři vyžaduje tzv. **Multi-Needle Reasoning** — schopnost současně nalézt, propojit a logicky syntetizovat několik vzájemně závislých informací (např. signaturu funkce v jednom modulu, její volání v druhém a konfigurační flag ve třetím). V takovém scénáři se naplno projevuje degradace pozornosti: s rostoucím objemem historie v kontextovém okně rapidně klesá spolehlivost křížového uvažování, což v dlouhých agentních sezeních vede k přehlédnutí okrajových podmínek a regresi v kódu.

2.3.5.5 KV Caching

KV caching slouží ke snížení výpočetní náročnosti modelu při generování tokenů. Místo toho, abychom při každém kroku inference znovu od začátku počítali pozornost pro celou dosavadní konverzaci, ponechává inferenční engine v paměti uložené již vypočtené vektory klíčů a hodnot (*Key-Value pairs*) a v každém kroku počítá a přidává pouze nově vygenerovaný token.

2.3.6 Prompt a kontextové inženýrství

Prompt engineering (inženýrství promptů) je disciplína zaměřená na systematický návrh, formulaci a optimalizaci textových instrukcí předkládaných modelu (8). V autonomních agentních systémech nepředstavuje prompt pouhou volnou konverzaci, ale slouží jako závazný kontrakt vymezující chování, práva a bezpečnostní mantinely agenta. Mezi klíčové techniky patří:

- **Systémový prompt (System Prompt):** Základní direktiva definující identitu agenta, dostupné nástroje a striktní provozní pravidla (např. pravidla pro zachování neměnnosti existujících testů, konvence pro formát commitů či zákaz destruktivních operací v repozitáři).
- **Few-shot prompting:** Technika vložení několika vzorových příkladů (vstup-výstup) přímo do kontextu. Tím model získá konkrétní představu o požadovaném schématu (např. syntaxi konfiguračního souboru `darkfactory.json`) bez nutnosti dodatečného dotrénování parametrů.
- **Chain-of-Thought (myšlenkový řetězec):** Vedení modelu k explicitní formulaci mezikroků a vnitřní dedukce před vygenerováním konečného kódu či akce. Rozklad komplexního zadání na postupné logické kroky zásadně potlačuje halucinace a tvoří základ fáze rozvahy (*Thought*) v cyklu ReAct.
- **Injekce dynamického kontextu:** Průběžné doplňování promptu o aktuální stav repozitáře, stromovou strukturu souborů, chybové výpisy kompilátoru a výsledky testů, díky čemuž agent operuje nad reálnými fakty namísto odhadů.

2.3.6.1 Úskalí negativních instrukcí a princip afirmativního vymezení

Zvláštní výzvou při formulaci systémových pravidel a zadání úkolů je definice omezujících podmínek pomocí zákazů a negativních instrukcí (např. „nikdy nemaž existující testy“, „neupravuj soubory mimo složku src“). V praxi je empiricky prokázáno, že jazykové modely negativní příkazy často porušují nebo zcela ignorují.

Tento jev má hluboké opodstatnění v samotném matematickém fungování mechanismu pozornosti transformeru:

1. **Pozornost zaměřená na přítomná slova:** Matice pozornosti (QK^T) vyhodnocuje sémantickou relevanci na základě **přítomných** tokenů. Zmínka zakázané operace (např. `test_suite.py` ve větě „Za žádných okolností nemaž test_suite.py“) aktivuje v embeddingovém prostoru silné vektorové asociace k danému souboru i k operaci smazání.
2. **Asymetrie negace v autoregresivní predikci:** Negace představuje logický operátor vysokého řádu vyžadující složenou kompozici významu. Při autoregresivním generování token po tokenu však model často podlehne silnější statistické asociaci mezi přítomnými sémantickými pojmy dříve, než uplatní logický kontext záporové částice.

V robustním inženýrství agentních systémů se proto uplatňují dva komplementární principy:

- **Afirmativní formulace pravidel:** Pravidla se namísto zákazů formulují pozitivním vymezením požadovaného chování a mantinelů (např. „Upravuj výhradně soubory deklarované v seznamu `target_files`“, „Před jakoukoli změnou ověř běh stávajících testů“).

- **Deterministická exekuční ochrana v harnessu:** Bezpečnostní mantinely a zákazy destruktivních operací nesmí záviset na poslušnosti jazykového modelu vůči promptu. Řídící harness systému DarkFactory proto prosazuje striktní omezení deterministicky na úrovni běhového prostředí — např. připojením chráněných cest v souborovém systému pouze pro čtení (*read-only mount*), sandboxováním exekuce terminálových příkazů a nezávislou validací změn v pull requestu před sloučením.

2.3.7 Agent vs Chatbot

Fundamentální rozdíl mezi chatbotem a agentem spočívá v rozsahu interakce s okolním světem: zatímco chatbot operuje výhradně v uzavřeném konverzačním rozhraní a generuje textové odpovědi, agentem se systém stává ve chvíli, kdy je vybaven sadou výkonných nástrojů (*tools*). Prostřednictvím nich dokáže aktivně zkoumat repozitář, vyhledávat informace na webu, modifikovat soubory a autonomně vykonávat příkazy v cílovém výpočetním prostředí.

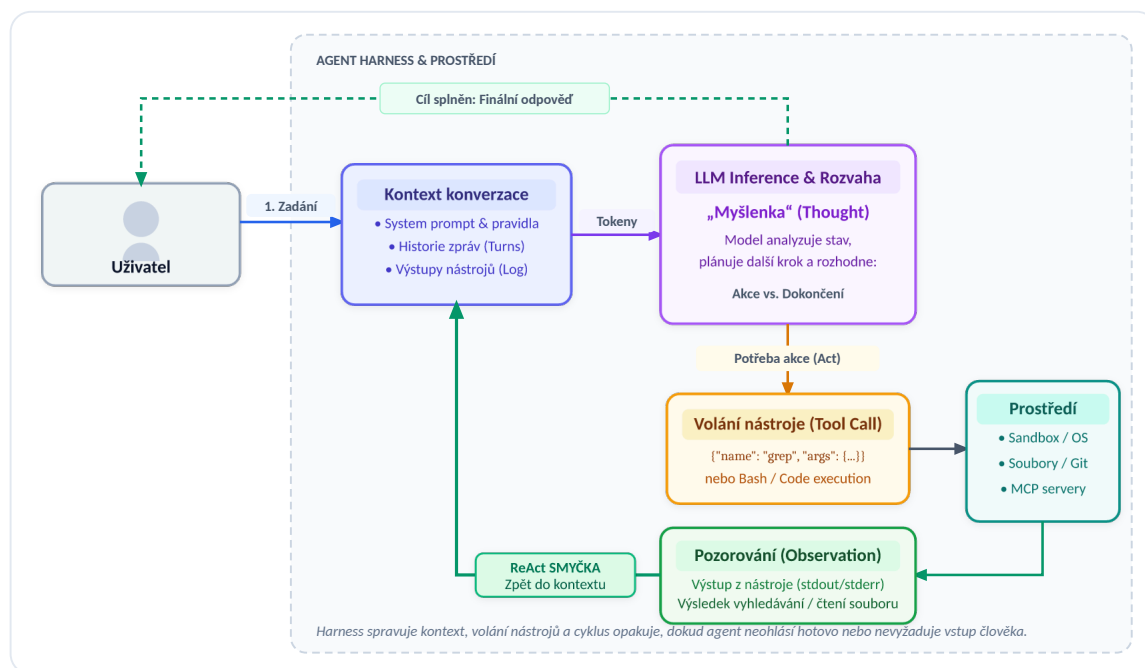
2.3.8 Harness a System Prompt

Harness je řídicí program obklopující jazykový model. Odlišuje se od inferenčního jádra (*inference engine*), které provádí samotné maticové výpočty sítě: harness má na starost rozhraní mezi uživatelem a agentem či chatbotem, předávání systémového promptu, bezpečné spouštění nástrojů a řízení iterativní smyčky neboli ReAct smyčky. Mezi typické příklady patří webová aplikace ChatGPT, terminálové rozhraní Claude Code, desktopová aplikace Codex a další agentní prostředí.

2.3.9 ReAct smyčka

Smyčka ReAct (*Reasoning and Acting*) tvoří základní stavební kámen, který z jazykového modelu vytváří autonomního agenta namísto pouhého pasivního generátoru textu (6). Princip nespočívá v pouhém přečtení vstupu a vykonání akce, nýbrž v soustavné alternaci vnitřní rozvahy (*Reasoning* neboli *Thought*) a vnějšího jednání (*Acting* neboli *Tool Call*). Model v každém kroku nejprve formuluje hypotézu či myšlenkový postup, na jehož základě provede cílenou akci.

Po vykonání akce harness připojí vrácený výsledek (*Observation*) do kontextového okna na konec logu konverzace. V navazujícím kroku inference model zhodnotí nový stav a cyklus opakuje, dokud úkol není vyřešen nebo dokud nerozhodne, že má dostatek podkladů pro předání finální odpovědi uživateli. Tímto způsobem je dosaženo adaptivního řešení problémů namísto jednorázové slepé generace.



Obrázek 2: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Diagram na Obrázek 2 znázorňuje základní iterativní cyklus moderních autonomních agentů. Po přijetí uživatelského zadání dochází v rámci inference k fázi rozvahy (*Reasoning*), kdy model formuluje vnitřní myšlenkový postup (*Thought*). Pokud je k vyřešení kroku zapotřebí externí akce, model vygeneruje strukturované volání nástroje (*Tool Call*). Řídící harness tento požadavek zachytí, bezpečně vykoná v cílovém prostředí (terminál, souborový systém, API či MCP server) a vrácený výsledek (*Observation*) připojí na konec kontextového okna. Celý aktualizovaný kontext je následně předložen modelu v další iteraci, dokud není úkol kompletní a výsledek předán uživateli. Tento princip byl formálně zaveden v práci *ReAct: Synergizing Reasoning and Acting in Language Models* ((6), [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)).

🔥 Hlubková kritika / Oponentura: Absence bezpečnostních pojištěk proti nekonečnému cyklu: Popis ReAct smyčky v sekci 2.3.8 uvádí, že cyklus se opakuje, dokud úkol není vyřešen nebo dokud model nerozhodne o předání odpovědi. V praxi však LLM velmi často upadají do perseverace a opakovaného volání téhož neúspěšného nástroje se stejnými parametry. Text postrádá teoretickou analýzu detekce uvíznutí (*stuck detection*), maximálního limitu tahů (*step budget*) a deterministického přerušení divergence ze strany řídicího harnessu.

2.3.10 Vyvolávání nástrojů

Abychom z jazykového modelu vytvořili autonomního agenta, musíme jej vybavit rozhraním pro interakci s okolním prostředím — tzv. nástroji (*tools*). V praxi existují dva převažující přístupy k realizaci nástrojů:

1. **Strukturované volání nástrojů (Function / Tool Calling v JSON):** Model ve svém výstupu zanechá strukturované volání odpovídající schématu zadanému v definici ná-

stroje. Řídicí harness toto volání zachytí, vykoná požadovanou funkci a vrátí výsledek zpět agentovi opět jako strukturovaná data. Tato možnost je spolehlivá pro jednoúčelové operace (např. integraci do podnikových systémů či zákaznické podpory — angl. *customer support*). Pro komplexní softwarový vývoj však představuje nevýhodu vysoká režie formátu JSON: velké množství formátovacích znaků se převádí na tokeny, což zbytečně plní kontextové okno a může vést k degradaci pozornosti (*Context Rot*).

2. **Přímé spouštění kódu (Code Execution):** Druhou možností je poskytnout agentovi plnohodnotné běhové prostředí (např. Bash, Python či TypeScript). Agent vygeneruje kód pro splnění svého záměru a harness jej spustí buď po jednotlivých příkazech, nebo jako ucelený skript. Z bezpečnostních důvodů je nezbytné, aby agent nepracoval přímo na nechráněném hostitelském počítači, nýbrž ve virtuálním prostředí odděleném bezpečnostní vrstvou — tzv. sandboxu (pískovišti). Tento způsob je pro softwarový vývoj nejefektivnější.

 **Hlubková kritika / Oponentura:** Iluzorní bezpečnost pískoviště: Druhý přístup (přímé spouštění kódu v kontejneru) práce nekriticky prezentuje jako „nejefektivnější“, zamlčuje však gigantická bezpečnostní a operační rizika. Běžný Docker kontejner není plnohodnotná bezpečnostní hranice (*security boundary*). Spuštění netestovaného kódu generovaného modelem s přístupem k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky tajných environmentálních proměnných (GitHub tokeny, API klíče k LLM poskytovatelům) přes postranní síťové kanály a nevratné poškození repozitáře. Práce postrádá jakoukoli analýzu formální izolace (např. microVM architektury jako AWS Firecracker, gVisor či striktní network namespaces) a nezodpovídá otázku, co zabráni kompromitovanému modelu zneužít CI runner jako odrazový můstek pro kompromitaci infrastruktury.

2.3.11 Dovednosti, skripty a záchytné body

Tyto standardy vznikly proto, aby vývojáři mohli modulárně upravovat chování agenta a rozšiřovat jeho schopnosti pro specifické doménové úlohy:

- **Dovednost (Skill):** Samostatný adresář sdružující instrukce, reference a pomocné soubory. Klíčovým prvkem je soubor `SKILL.md`, který využívá hlavičku v metadatovém formátu YAML frontmatter (ohraňovanou trojicí pomlček `---`). V hlavičce je definován název a stručný popis dovednosti. Řídicí harness do základního systémového promptu vkládá pouze tato metadata; samotný text podrobného návodu se do kontextu načte až ve chvíli, kdy agent danou dovednost vyvolá. Tím se efektivně šetří kapacita kontextového okna.
- **Skript (Script):** Jednoúčelový spustitelný program (např. v jazyce Python či Bash). Skripty jsou pro efektivitu práce vitální: agent nemusí generovat každý příkaz z paměti, ale spouští deterministické a otestované postupy pro analýzu a transformaci kódu.

- **Záchytný bod (Hook):** Skript, který se v harnessu automaticky vyvolá při určité systémové události či stavovém přechodu (např. při inicializaci sezení, před odesláním promptu modelu nebo při selhání nástroje). Hooky umožňují vynucovat bezpečnostní pravidla a logování nezávisle na vůli samotného modelu.

2.3.12 MCP Server

Model Context Protocol vznikl v laboratořích společnosti Anthropic jako standardizovaný způsob pro efektivní interakci agentů s API servery. Ve své podstatě se jedná o nástroje postavené na rozhraní externího API, tudíž protokol sjednocuje různé standardy do jediného otevřeného rozhraní a přidává kontext — metadata pro každou přítomnou metodu, aby agent předem věděl, co od ní očekávat, a nemusel sám odhadovat fungování backendu. Praktický význam specifikace Model Context Protocol (9) spočívá ve sjednocení dříve fragmentovaného ekosystému proprietárních rozhraní a jednoúčelových integračních skriptů. Řídící harness se díky standardu stává univerzálním klientem a veškeré externí schopnosti, datové zdroje či nástroje se integrují jako samostatné procesy — tzv. MCP servery. Komunikace probíhá přes standardizovaný protokol JSON-RPC, a to buď lokálně prostřednictvím standardního vstupu a výstupu (`stdio`), nebo vzdáleně pomocí protokolu HTTP se Server-Sent Events (`SSE`). Tento modulární přístup striktně odděluje běhové prostředí agenta od samotné implementace nástrojů: libovolnou schopnost (např. přístup k databázi, vyhledávání v repozitáři nebo správu GitHub úkolů) stačí naimplementovat jednou a lze ji okamžitě zpřístupnit jakémukoli kompatibilnímu agentovi bez nutnosti zásahu do kódu samotného harnessu.

2.3.13 Meta Harness

V reálném světě se ukázalo, že nejefektivnější harness je takový, který dá samotnému agentovi rozhodovací pravomoc nad vlastní architekturou a umožňuje kontinuální změnu, tedy samořízenou evoluci (10). Tento přístup je mimořádně efektivní pro maximalizování pracovní kapacity agenta, v praxi se však hodí především pro osobního agenta. U komerčních nasazení, jako je zákaznická podpora (*customer support*), provozovatel naopak striktně vyžaduje deterministické mantinely a vylučuje, aby model modifikoval své vlastní provozní instrukce.

2.3.14 Subagenti

Když je agent vybaven nástrojem umožňujícím vyvolat další specializovanou instanci jazykového modelu, zadat jí dílčí úlohu, asynchronně s ní komunikovat a sledovat její postup, efektivita řešení rozsáhlých softwarových problémů dramaticky roste. V teorii autonomních systémů se tyto delegované entity označují jako subagenti (*subagents*).

Tato architektura umožňuje hierarchickou dekompozici problému: hlavní koordinační agent (*orchestrator*) udržuje globální strategii a pověřuje úzce profilované subagenty

izolovanými činnostmi — například řešerší dokumentace, prozkoumáním rozsáhlého adresářového stromu repozitáře nebo syntaktickou opravou konkrétního modulu. Zásadní architektonickou výhodou je ochrana a izolace kontextového okna: rozsáhlý a výpočetně náročný průzkumný kontext subagenta se po dokončení úkolu zahodí a rodičovskému orchestrátoru je předán pouze syntetizovaný, čistý výsledek. Tím se zabráňuje zahlcení primárního kontextu (*context pollution*) a degradaci kognitivních schopností hlavního agenta.

2.3.15 Workflows neboli Graph Engineering

Jakmile se snažíme organizovat nebo automatizovat úlohu složitější než několik přímočarých kroků, lineární sekvence promptů přestává stačit. V takovém okamžiku přicházejí na řadu pracovní postupy (*workflows*), v teoretické informatice reprezentované jako grafy. Grafy se v softwarovém vývoji používají v mnoha situacích. Graf se skládá z uzlů (*nodes*) a hran (*edges*) — v kontextu agentních systémů každý uzel představuje izolovaného agenta s připraveným systémovým promptem, dovednostmi a nástroji, zatímco spoje mezi nimi vymezují tok informací a posloupnost práce. Abstrakcí tohoto grafového systému jsme schopni tuto pipeline nasadit na libovolnou úlohu, a to i dynamicky za pomoci orchestrace nadřazeným agentem.

Tento koncept se v informatice označuje jako orientovaný acyklický graf (**DAG** — *Directed Acyclic Graph*). Uzly grafu představují samostatné, specializované výpočetní kroky či izolované agenty, zatímco orientované hrany určují pořadí závislostí a tok kontextových informací. V moderních orchestrátorech a CI/CD platformách (zejména v GitHub Actions) se závislosti mezi úlohami deklarují pomocí direktivy `needs: [...]`.

Jak podrobně rozvádí praktická část této práce na architektuře systému DarkFactory, celý životní cyklus automatizovaného požadavku je strukturován právě jako DAG složený z pěti klíčových fází:

1. **Detekce a kontext:** rozpoznání prostředí projektu, načtení konfiguračního souboru `darkfactory.json` a příslušných systémových pravidel.
2. **Interpretace a plánování:** formulace přesného technického postupu a vytvoření plánovacího úkolu dříve, než dojde k samotnému zásahu do kódu.
3. **Implementace (kódování):** spuštění agenta s přesně vymezenou sadou nástrojů v izolované větvi repozitáře.
4. **Verifikace a testování:** automatické sestavení projektu, spuštění testovací sady a kontrola kvality změn.
5. **Schvalovací brána (Governance):** podmíněné zastavení toku grafu a vyžádání lidské kontroly před finálním sloučením pull requestu.

Zásadní předností grafového uspořádání je determinismus a striktní bezpečnost: pokud kterýkoli uzel selže (např. automatizované testy detekují regresi), exekuce se v dané větvi

grafu okamžitě přeruší a navazující kroky se nespustí, což zabraňuje poškození hlavní vývojové linie.

2.4 Human in the loop neboli člověk ve smyčce

Čím je systém samostatnější, tím důležitější je otázka, kde do procesu vstupuje člověk. Úplná autonomie není cílem; cílem je autonomie v rutinních krocích a lidské rozhodnutí tam, kde je nevratné nebo kde chybí měřítko správnosti.

2.4.1 Schvalovací body

Schvalovací bod je místo, kde se proces zastaví a čeká na potvrzení. Jeho umístění je kompromisem: příliš mnoho schvalování popírá smysl automatizace, příliš málo znamená ztrátu kontroly. Osvědčeným řešením je schvalovat *záměr* (co se má udělat) a *plán* (jak se to má udělat) dříve, než vznikne kód.

2.4.2 Dohledatelnost

Aby byla automatizovaná změna přezkoumatelná, musí být zřejmé, z jakého požadavku vzešla. Uchování doslovného znění původního zadání je proto součástí návrhu, nikoli formalitou: parafráze ztrácí význam, který do zadání vložil ten, kdo je formuloval.

2.4.3 Eskalace a transparence selhání

Systém, který své vlastní selhání zamlčí nebo zamete pod koberec, je nebezpečnější než systém, který zjevně havaruje: nespolehlivost se v něm stává neviditelnou a postupně eroduje důvěru uživatele v celý autonomní provoz. V konceptu člověka ve smyčce proto hlášení chyb a eskalace selhání netvoří pouhý doplňkový technický detail, nýbrž fundamentální bezpečnostní pilíř. Pokud autonomní agent narazí na vyčerpání kontextového okna, syntaktickou chybu neřešitelnou v rámci rozpočtu tahů nebo selhání integračních testů, nesmí skončit tichým uváznutím či nekonečným cyklem. Místo toho musí harness deterministicky zachytit chybový stav, sestavit strukturovaný diagnostický protokol (obsahující chybový stack trace, diff provedených změn a stav kontextu) a srozumitelně jej eskalovat člověku formou notifikace či dedikovaného incidentu. Tím je zajištěna plná observabilita a okamžitá lidská dohledatelnost.

3 Praktická část

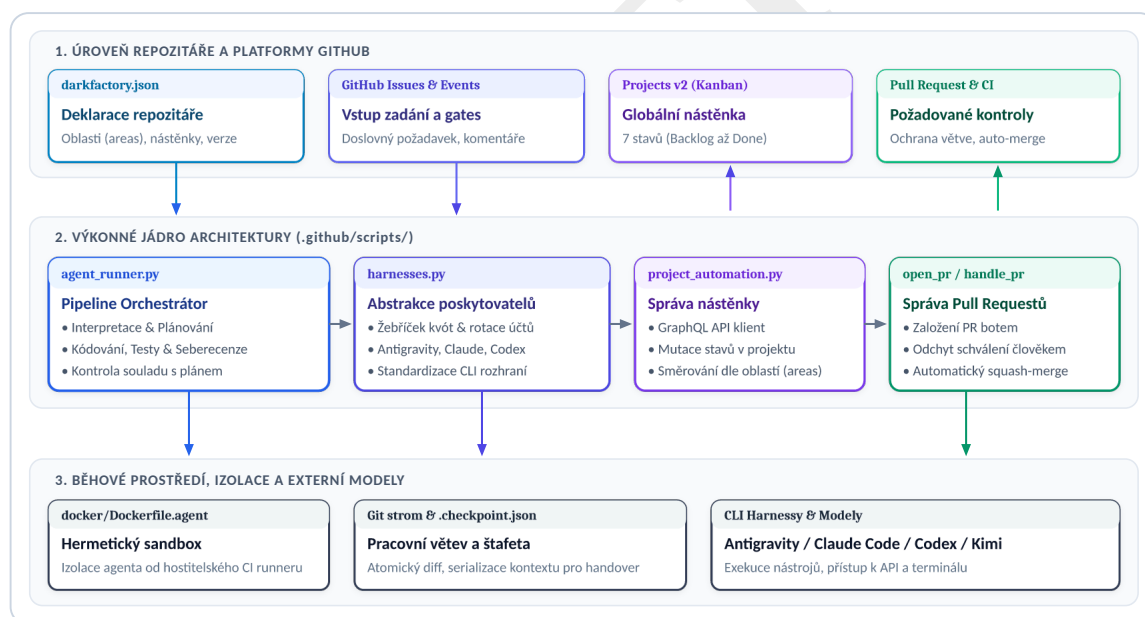
Praktickou částí práce je systém **DarkFactory** — sada pravidel, skriptů a pracovních postupů, které z repozitáře udělají samostatně pracující vývojový provoz. Zdrojový kód je veřejně dostupný (11).

3.1 Cíl a rozsah systému

Systém má převzít rutinní kroky vývojového procesu: přijetí požadavku, jeho interpretaci, naplánování, provedení změny a její ověření. Nemá nahradit rozhodování o tom, co se má stavět; to zůstává člověku, a systém je navržen tak, aby si toto rozhodnutí vyžádal dříve, než začne pracovat.

3.2 Architektura

Systém DarkFactory je navržen jako modulární stavebnice složená z řídicích skriptů v jazyce Python, šablon pracovních postupů pro GitHub Actions a hermetického* kontejnerového prostředí. Architektura striktně odděluje deklarativní konfiguraci konkrétního repozitáře od samotné logiky orchestrace.



Obrázek 3: Dekompozice komponent systému DarkFactory: třívrstvá architektura propojující platformu GitHub, výkonné skriptové jádro a izolované běhové prostředí s modely.

Celý systém je dekomponován do tří funkčních vrstev znázorněných na Obrázek 3:

- 1. Vrstva deklarace a platformy GitHub:** Zahrnuje centrální konfigurační soubor `darkfactory.json` jako jediný zdroj pravdy pro daný repozitář, rozhraní GitHub Issues

*Pojem **hermetické prostředí** (angl. *hermetic environment*) označuje v softwarovém inženýrství takové výpočetní a běhové prostředí, které je zcela izolované od nekontrolovaných stavů hostitelského operačního systému a okolní sítě. Veškeré nástroje, knihovny a systémové závislosti jsou v něm explicitně deklarovány a uzamčeny na konkrétních verzích, což zaručuje absolutní determinismus a reprodukovatelnost: proces spuštěný v hermetickém kontejneru skončí vždy identickým výsledkem bez ohledu na to, kde a kdy byl vyvolán.

pro zadávání požadavků v doslovném znění a projektovou nástěnku GitHub Projects v2, která vizualizuje stav pipeline napříč všemi zapojenými repozitáři v sedmi stavech (od *Backlog* po *Done*).

2. **Výkonné jádro pipeline (`.github/scripts/`):** Tvoří jej sada úzce spolupracujících skriptů v jazyce Python. Klíčovým prvkem je `agent_runner.py` jako hlavní stavový automat řídící životní cyklus požadavku. Doplnuje jej `harnesses.py` pro sjednocení rozhraní různých modelů (Antigravity, Claude Code, Codex, Kimi) a rotaci kvótových účtů, `project_automation.py` pro obousměrnou synchronizaci stavů přes GraphQL rozhraní a skripty `open_pr.py` spolu s `handle_pr_approval.py` zajišťující automatické vystavení pull requestu botem a následný bezpečný squash-merge po schválení člověkem.
3. **Izolované běhové prostředí a nástroje:** Zahrnuje kontejnerový sandbox (`docker/Dockerfile.agent`), který hermeticky izoluje běh agenta od hostitelského CI runneru. Agent operuje v izolované větvi repozitáře a veškerý postup je průběžně atomic-ky serializován do kontrolních bodů (`.checkpoint.json`), což umožňuje bezetrátové předání štafety mezi různými poskytovateli.

Návrh vychází z jednoho základního požadavku: pracovní postupy i skripty musí být ve všech repozitářích totožné. Kdyby se kopírovaly, začaly by se rozcházet, a oprava chyby by se musela provádět tolikrát, kolik je repozitářů.

3.2.1 Sdílení místo kopírování

Pracovní postupy jsou proto *volané*, nikoli kopírované. Repozitář, který systém používá, obsahuje pouze krátký soubor odkazující na sdílený pracovní postup připnutý ke konkrétní verzi. Připnutí je podstatné: bez něj by se změna sdíleného postupu okamžitě promítla do všech repozitářů, včetně těch, které na ni nejsou připraveny.

 **Strukturální upozornění:** Strukturální stručnost a chybějící konkrétní příklad volání: Podkapitola 3.2.1 sestává z jediného odstavce o šesti řádcích. Doporučuji doplnit konkrétní ukázkou zdrojového kódu volajícího workflow (např. direktivu `uses: marius-patrik/DarkFactory/.github/workflows/ci.yml@...`) pro názornou demonstraci principu a způsobu předávání parametrů.

3.2.2 Jediný konfigurační soubor

Vše, co se mezi repozitáři liší, je soustředěno do jediného souboru `darkfactory.json`. Ten popisuje totožnost repozitáře, jeho oblasti, nástěnky, na které patří jeho úkoly, a případné odchylky od výchozího chování. Sdílený postup je díky tomu ve všech repozitářích shodný bajt po bajtu.

```

{
  "identity": {
    "owner": "marius-patrik",
    "repo": "DarkFactory",
    "default_branch": "darkfactory"
  },
  "versioning": {
    "mode": "semver",
    "tag_prefix": "v",
    "initial": "0.1.0"
  },
  "areas": {
    "$default": "ci",
    "agents": {
      "description": "Orchestration, personas, adapters",
      "keywords": ["agent", "harness", "persona", "prompt"]
    },
    "governance": {
      "description": "Branch protection, required checks",
      "keywords": ["governance", "rule", "protection", "board"]
    },
    "ci": {
      "description": "GitHub Actions workflows, runner scripts",
      "keywords": ["ci", "action", "workflow", "docker"]
    }
  },
  "board": {
    "global_title": "Global",
    "link_boards": ["Global", "DarkFactory", "OdbornaPrace",
      "ChessWithQuests"]
  }
}

```

Výpis 1: Ukázka konfigurace v souboru `darkfactory.json` vymezující identitu repozitáře, doménové oblasti (`areas`) a propojení s projektovými nástěnkami.

Jak je patrné z Výpis 1, sekce `areas` slouží jako jednotný zdroj pravdy pro štitkování úkolů i směřování agentů podle shody klíčových slov. Sekce `board` pak zajišťuje agregaci do globální i repozitářové GitHub Projects nástěnky bez nutnosti manuální konfigurace v samotných workflow.

⚠ Chyba / Nesrovnalost k opravě: Nepřesná cesta ke konfiguračnímu manifestu: Text v sekcích 3.2 a 3.2.2 hovoří o souboru `darkfactory.json` v kořeni repozitáře, avšak v reálné implementaci systému DarkFactory (i v Příloze B) je manifest striktně umístěn v konfiguračním adresáři `.github/darkfactory.json` (`MANIFEST_PATH = os.path.join(".github", "darkfactory.json")`). Je vhodné cestu v kapitole 3 sjednotit na `.github/darkfactory.json`.

3.3 Životní cyklus požadavku

Požadavek prochází systémem v pevně daných krocích, mezi nimiž jsou striktně vyžadovány schvalovací body pro udržení lidské kontroly nad rozsahem i kvalitou implementace.



Obrázek 4: Stavový diagram životního cyklu požadavku v systému DarkFactory: přechody mezi stavy od založení issue přes schvalovací brány (Human Gates), tvorbu větve a PR až po automatické sloučení a uzavření úkolu.

Jednotlivé kroky životního cyklu znázorněné na Obrázek 4 na sebe navazují v přesně daném pořadí:

- 1. Zadání úkolu:** Uživatel založí požadavek (GitHub Issue) s doslovným zněním svého zadání.
- 2. Interpretace:** Systém požadavek analyzuje (`interpreted`) a předloží své porozumění rozsahu.
- 3. Human Gate 1 (Schválení záměru):** Člověk potvrdí, že systém pochopil cíl správně.
- 4. Návrh plánu:** Po schválení vzniká podřízený úkol (`planned`) s technickým rozpisem kroků.
- 5. Human Gate 2 (Schválení plánu):** Člověk schválí technický plán dříve, než dojde k zásahu do kódu.

6. **Kódování a verifikace:** Systém vytvoří větev (`branch created`), agent napíše kód i testy a provede vlastní přezkoumání a ověření proti plánu.
 7. **Návrh změny (`PR open`):** Bot otevře pull request a spustí integrační CI kontroly.
 8. **Human Gate 3 a dokončení:** Po schválení pull requestu člověkem systém provede automatické sloučení (`auto-merge`), smaže větev a uzavře rodičovský úkol.
- Doslovné znění zadání je uchováno záměrně. Zkušenost z vývoje ukázala, že právě parafráze bývá zdrojem nedorozumění: shrnutí požadavku se zdá výstižné tomu, kdo je psal, a přitom už neobsahuje to, na čem zadavateli záleželo.

 **Hlubková kritika / Oponentura:** Pochybná autonomie a paralýza lidským faktorem: Název „DarkFactory“ evokuje bezobslužnou továrnu (angl. *lights-out manufacturing*), která běží plně autonomně bez lidského zásahu. Zavedení tří synchronních schvalovacích bran (Human Gate 1: záměr, Human Gate 2: plán, Human Gate 3: PR) však z procesu činí silně blokující workflow. Pokud musí člověk manuálně schválit každou fázi drobného úkolu, tráví agent 95 % životního cyklu čekáním na lidskou reakci. Skutečná průchodnost systému je pak determinována latencí člověka, nikoli rychlostí modelů. Práce navíc ignoruje fenomén únavy ze schvalování (*review fatigue*), kdy člověk po desítkách syntetických notifikací rezignuje na důkladnou kontrolu a začne plány i diffy schvalovat mechanicky bez čtení.

3.4 Popis prostředí repozitáře

Aby systém věděl, které úlohy má spustit, musí rozpoznat, z čeho se repozitář skládá. Rozpoznávání vychází z názvů souborů popisujících balíček; jejich přítomnost je spolehlivější než jakýkoli ruční záznam, protože se nemůže rozejít se skutečností.

Soubor	Prostředí	Doména
<code>pyproject.toml</code>	Python	kód
<code>Cargo.toml</code>	Rust	kód
<code>package.json</code>	Node	kód
<code>go.mod</code>	Go	kód
<code>typst.toml</code>	Typst	text
<code>.latexmkrc</code>	LaTeX	text
<code>lakefile.toml</code>	Lean	matematika

Tabulka 3: Rozpoznávání prostředí podle souboru popisujícího balíček.

3.4.1 Domény a prostředí

Rozlišení dvou úrovní se ukázalo jako nutné až v průběhu práce. *Prostředí* říká, jaký nástroj je potřeba; *doména* říká, jakému způsobu řízení výsledek podléhá. Python a Rust jsou dvě prostředí téže domény — kód se testuje a balí. Typst a LaTeX jsou dvě prostředí jiné domény — text se sází a publikuje.

Toto rozlišení dovoluje popsat i repozitář, který obsahuje zároveň program a text — jako právě tato práce, jejíž praktickou částí je software popisovaný v Tabulka 3. Bez něj by bylo nutné buď považovat sazbu za zvláštní případ kódu, nebo pro texty vytvořit samostatný systém.

V případě matematické domény a prostředí Lean (reprezentovaného manifestem `lakefile.toml` v Tabulka 3) se v aktuální implementaci jedná o deklarativní přípravu a taxonomické vymezení v subsystému `environment.py`. Plná integrace interaktivního dokazovače Lean 4 do CI kontejneru je plánována jako další směr rozvoje, neboť vyžaduje specifický toolchain `elan` a vysoké výpočetní nároky na strojovou verifikaci důkazů.

3.4.2 Deklarace jako doplněk rozpoznávání

~~Rozpoznávání nemůže vidět všechno. Repozitář této práce například přibaluje vlastní písma, takže příkaz pro sazbu není výchozí. Konfigurační soubor proto umožňuje výchozí chování přepsat, aniž by bylo nutné vypnout rozpoznávání jako celek.~~ Rozpoznávání prostředí nemůže předvídat veškeré specifické požadavky projektu. Repozitář této práce například přibaluje vlastní sadu písem v podadresáři `fonts/`, takže příkaz pro sazbu vyžaduje dodatečný přepínač `--font-path fonts`. Konfigurační soubor `darkfactory.json` proto umožňuje výchozí parametry transparentně rozšířit nebo přepsat, aniž by bylo nutné vypnout automatické rozpoznávání prostředí jako celek:

```
[
  "identity": { "name": "odborna-prace", "domain": "text" },
  "build": {
    "typst": {
      "entrypoint": "main.typ",
      "output": "out/main.pdf",
      "extra_args": ["--font-path", "fonts"]
    }
  }
]
```

Výpis 2: Ukázka deklarativního přepsání parametrů sestavení v souboru `darkfactory.json`.

Jak ukazuje Výpis 2, systém zkombinuje detekované prostředí s explicitní specifikací z manifestu. CI úloha tak automaticky zkonstruuje přesný příkaz `typst compile --font-path fonts main.typ out/main.pdf` bez nutnosti manuálního zásahu do sdíleného kódu akce.

3.5 Orchestrace napříč poskytovateli

Systém neváže na jednoho poskytovatele modelu. Úkol je popsán způsobem nezávislým na rozhraní a předán prvnímu dostupnému poskytovateli; vyčerpá-li tento kvótu, předá se tentýž úkol dalšímu v pořadí, místo aby se proces zastavil.

Tato vlastnost byla přímou reakcí na provozní zkušenost: zastavení celého procesu kvůli vyčerpané kvótě jediného poskytovatele bylo nejčastější příčinou prostoje.

Mechanika předávání štafety (*baton handover*) probíhá zcela pod kontrolou nadřazeného procesu (`agent_runner`). Jakmile volající skript zachytí vyčerpání limitů (např. HTTP kód 429 nebo chybový stav `RESOURCE_EXHAUSTED`), okamžitě zmrazí aktuální běh a provede atomickou serializaci stavu:

1. **Pracovní strom v Gitu:** Veškeré rozpracované úpravy v souborovém systému jsou uloženy do pracovní větve a vytvoří se kontrolní otisk (`git diff`).
2. **Serializace kontextu a štafety:** Log konverzace (předchozí kroky, volání nástrojů i vrácená pozorování) je spolu s metadaty úkolu zapsán do strukturovaného kontrolního bodu (*checkpoint* ve formátu JSON).
3. **Rotace po žebříčku kapacit:** Runner neponižuje model na slabší variantu v témže fondu (což by novou kapacitu nepřineslo), nýbrž rotuje účty, oddělené kvótové fondy (*quota pools*) nebo přepne na záložní CLI harness (např. AntigraVity, Claude Code či Codex).
4. **Rekonstituce a navázání:** Náhradní agent obdrží serializovanou štafetu, jeho adaptér přeloží historii kroků do nativního formátu nového poskytovatele a ověří stav repositáře. Běh plynule naváže v přesném bodě přerušení, aniž by došlo ke ztrátě dosavadní práce či kontextu.

 **Hlubková kritika / Oponentura:** Kritická slepá skvrna v heterogenní štafetě: Představa, že odlišný model (např. Claude po AntigraVity či Codexu) plynule naváže na rozpracovanou práci pouhým načtením serializovaného logu a `git diff`, zamlčuje zásadní problém nekompatibility vnímání kontextu (*prompt sensitivity*). Každá modelová rodina vyžaduje diametrálně odlišný formát nástrojů, odlišně reaguje na systémový prompt a jinak interpretuje mezivýsledky. V reálném provozu vede synteticky přeložená historie často k okamžité dezorientaci nového modelu, opakování již hotových kroků nebo halucinaci neexistujících nástrojů. Práce neobsahuje žádné empirické vyhodnocení úspěšnosti štafety: Kolik úloh po předání štafety skutečně úspěšně dobehlo a v kolika procentech případů vedla rotace k havárii a divergenci kontextu?

 **Návrh na vylepšení:** Správa tajemství a rotace tokenů: Doporučuji v sekci 3.5 stručně specifikovat bezpečnostní model správy API klíčů při rotaci poskytovatelů — jakým způsobem jsou klíče (Anthropic, OpenAI, Google) bezpečně předávány do kontejneru a jak je zamezeno jejich nechtěnému zápisu do kontrolních bodů (`.checkpoint.json`) či veřejných CI logů.

3.6 Ověřování změn

Každá změna musí projít požadovanými kontrolami. Ty jsou navrženy tak, aby vždy skončily nějakým výsledkem — úloha, která se může „přeskočit“, by jinak zablokovala slučování napořád, jak bylo vysvětleno v kapitole 2.

```
paper:
  steps:
    - name: Detect the paper domain
      id: detect
    - name: Typeset with Typst
      if: steps.detect.outputs.typst == 'true'
    - name: No paper in this repository
      if: steps.detect.outputs.present != 'true'
    run: echo "nothing to typeset."
```

Výpis 3: Zjednodušená úloha, která vždy skončí výsledkem.

Struktura v Výpis 3 je pro celý systém typická: nejprve se zjistí, zda je v repozitáři co dělat, a teprve pak se pracuje. Poslední krok zajišťuje, že úloha skončí úspěchem i tam, kde není co ověřovat.

3.6.1 Optimalizace běhu a cachování závislostí

Pro udržení nízké latence vývojového cyklu a rychlou zpětnou vazbu agentovi využívá systém DarkFactory agresivní cachování balíčků a závislostí napříč jednotlivými integračními běhy. V rámci GitHub Actions se využívá mechanismus `actions/cache` provázaný s kryptografickým hashem souborů specifikujících uzamčené verze závislostí (např. `uv.lock`, `Cargo.lock` či `package-lock.json`).

Při opětovném spuštění CI workflow pro nově vytvořený commit se balíčky nestahují z externích registrů (PyPI, crates.io, npm), nýbrž se obnoví z lokální mezipaměti běžce během 2–4 sekund. U jazyka Python systém navíc standardně využívá maticové sestavení (`matrix.python-version: ["3.11", "3.12"]`), čímž je zaručena kompatibilita napříč podporovanými verzemi interpretu v paralelních úlohách bez lineárního prodloužení celkového času běhu.

3.6.2 Automatické hlášení a deduplikace selhání

~~Selhání kterékoli úlohy zakládá úkol s odkazem na neúspěšný běh. Opakované selhání téže úlohy nezakládá další úkol, nýbrž doplní komentář ke stávajícímu; bez toho by úloha selhávající při každé změně zahrtila seznam úkolů. Jakmile úloha znovu uspěje, úkol se sám uzavře.~~ Pokud kterákoli integrační či verifikační úloha v CI selže, systém automaticky vyvolá záchranné workflow `report-failure.yml`. To v klientském repozitáři založí nový incident ve formě GitHub Issue s detailním označením selhaného kroku a přímým odkazem na protokol neúspěšného běhu.

Zásadním prvkem architektury je inteligentní deduplikace incidentů: opakované selhání téže úlohy (např. při následném nepovedeném commitu) nezakládá nový redundantní úkol, nýbrž identifikuje stávající otevřený incident a pouze k němu připojí nový diagnostický komentář s aktuálním logem. Tím se spolehlivě zamezuje zahlcení projektové nástěnky desítkami duplicitních hlášení. Jakmile je chyba v kódu odstraněna a navazující běh úspěšně projde všemi kontrolami, systém otevřený incident v GitHub Issues automaticky uzavře s odkazem na opravný commit.

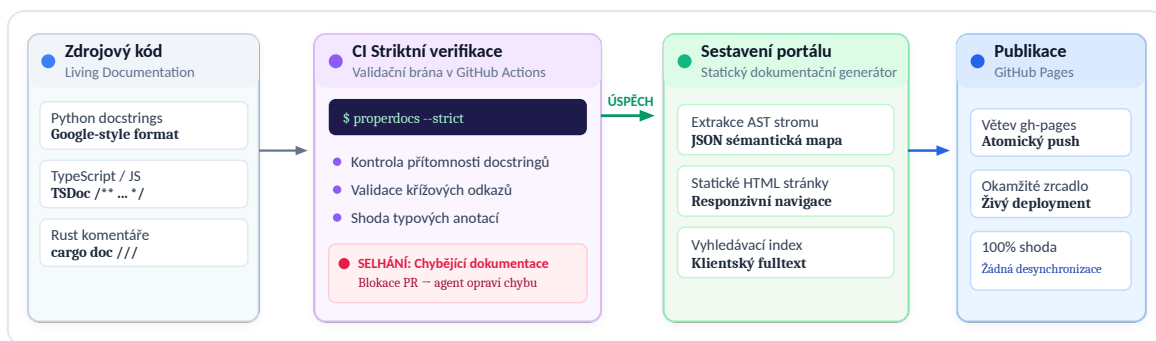
3.7 Automaticky generovaná dokumentace

V moderním softwarovém vývoji představuje manuální údržba dokumentace permanentní zdroj chyb a desynchronizace: jakmile se kód vyvíjí rychleji než textové popisy, dokumentace se nevyhnutelně stává zastaralou a nespolehlivou. Systém DarkFactory proto prosazuje striktní princip stoprocentního odvozování veškeré projektové a API dokumentace přímo ze zdrojového kódu a strukturovaných inline komentářů (tzv. *living documentation*). Tento přístup využívá přirozené standardy konkrétních programovacích jazyků a prostředí:

- **Python:** Dokumentační komentáře v konvenci Google-style docstrings, z nichž generátor automaticky extrahuje typy parametrů, návratové hodnoty a signatury výjimek.
- **TypeScript / JavaScript:** Standardizované bloky TSDoc (`/** ... */`) anotující rozhraní, typové definice a exportované funkce.
- **Rust:** Integrované dokumentační komentáře `///` kompilované nativním nástrojem `cargo doc`.

Zásadní inovací je zařazení striktní verifikace do každého integračního běhu v CI (např. voláním `properdocs build --strict` či odpovídajících linterů dokumentace). Jakýkoli chybějící docstring nově přidané funkce, syntaktická chyba v parametrech nebo neplatný křížový odkaz na neexistující symbol způsobí okamžité selhání validační úlohy v CI a zablokuje sloučení změny. Když autonomní agent implementuje nový modul, je tímto mechanismem deterministicky nucen vytvořit a aktualizovat i kompletní dokumentační anotace.

Po úspěšném schválení pull requestu člověkem a jeho sloučení do hlavní větve centrální pracovní postup automaticky sestaví statickou podobu dokumentačního portálu a publikuje jej na GitHub Pages. Tím odpadá jakákoli manuální údržba externích zrcadel a dokumentace zůstává v absolutní shodě s reálným stavem kódu.



Obrázek 5: Procesní schéma generování a striktní validace živé dokumentace v CI pipeline.

3.8 Systém revizních značek pro lidský dohled nad textem

Při rozšiřování systému DarkFactory na správu a tvorbu dokumentace (doména textu) vyvstala potřeba formalizovat spolupráci člověka a autonomního agenta přímo v sazebním formátu Typst. Výsledkem je protokol vizuálních revizních značek (**Review Markers**) a textových revizních funkcí, který barevně a sémanticky rozlišuje stav zpracování jednotlivých pasáží:

💡 Návrh na vylepšení: Konstruktivní doporučení, nápady na rozšíření, doplnění schémat či návrhy na praktické propojení. Po zapracování se panel smaže.

⚠️ Chyba / Nesrovnalost k opravě: Detekované věcné nepřesnosti, logické mezery, překlepy nebo duplicita obsahu. Značka přesně formuluje vadu a zaniká s jejím odstraněním.

🔍 Strukturální upozornění: Upozornění na hloubkovou nevyváženost kapitol, chybějící dekompozice komponent či nesoulad s osnovou práce.

🔥 Hloubková kritika / Oponentura: Hloubková teoretická a architektonická oponentura bez servítků — odhalování slepých míst, neověřených předpokladů, bezpečnostních rizik a metodologických slabín formulovaných jako břitké otázky k obhajobě.

V toku textu se uplatňují tyto zvýrazňovací a srovnávací funkce:

- **Žluté zvýraznění** (`#draft[...]` / `#unconfirmed[...]`): Označuje neověřený text konceptu čekající na autorské posouzení a revizi.
- **Zelené zvýraznění** (`#added[...]`): Označuje nově přidaný text vygenerovaný autonomním agentem na základě požadavku či doporučení.
- **Modré zvýraznění** (`#confirmed[...]`): Označuje text potvrzený uživatelem, který dosud neprošel finální integrací.
- **Červené zvýraznění s přeškrtnutím** (`#removed[...]`): Označuje text navržený k odstranění z rukopisu.

- ~~Původní nahrazovaný text~~ Srovnávací diff (`#diff(old, new)`): Zobrazuje původní text přeškrtnutý v červené barvě následovaný novým textem v zelené barvě.
- Čistý neoznačený text představuje finální, autorsky schválený a přijatý text v hlase autora (v rozpracovaném stavu konceptu jsou veškeré dosud neuzavřené pasáže zviditelněny revizními funkcemi).

Tento protokol umožňuje autonomnímu agentovi navrhopat změny s transparentním vy-
značením míry jistoty a člověku poskytuje okamžitou vizuální kontrolu nad tím, které části
rukopisu již prošly lidskou redakcí a které ještě čekají na posouzení.

KONCEPT

4 Výsledky a diskuse

4.1 Profil nasazených repozitářů a testovací prostředí

System byl nasazen na tři repozitáře různé povahy: na vlastní repozitář systému, na aplikaci psanou v několika jazycích a na menší projekt. Sledovány byly tři veličiny: počet vlastních skriptů a pracovních postupů v jednotlivých repozitářích, počet řádků odstraněného duplicitního kódu a chování systému v provozu.

Pro praktické ověření funkčnosti, robustnosti a přenositelnosti systému DarkFactory bylo zvoleno testovací prostředí skládající se ze tří produkčních repozitářů s odlišnou architekturou, programovacími jazyky a rozsahem:

1. **DarkFactory** (mateřský repozitář): Centrální vývojové prostředí řídicího metaharnessu a sdílených GitHub Actions. Kódová báze je postavena na jazyce Python 3.12 s využitím balíčkovacího nástroje uv, striktního typování mypy, formátování ruff a testovacího rámce pytest.
2. **OdbornaPrace-paper**: Repozitář samotného rukopisu této práce reprezentující doménu textu a dokumentace. Kódová báze kombinuje sazbu v moderním typografickém systému Typst, doprovodné obslužné a validační skripty v Pythonu a plně automatizovanou kompilaci a publikaci PDF na GitHub Pages.
3. **ChessWithQuests**: Menší aplikační projekt implementující šachovou herní logiku a pravidla. Slouží k ověření chování pipeline při izolovaných, algoritmicky ohraničených úlohách s rychlou zpětnou vazbou jednotkových testů.

V rámci sledovaného období byly kvantitativně i kvalitativně monitorovány čtyři klíčové dimenze provozu: architektonická konsolidace a údržbová zátěž, spolehlivost a průchodnost agentních běhů, časová latence jednotlivých fází životního cyklu a chování záchranných mechanismů při chybách a výpadech.

4.2 Sjedení pracovních postupů

Základním architektonickým principem systému DarkFactory je sdílení pracovních postupů namísto jejich ad-hoc kopírování do jednotlivých projektů. Klientské repozitáře (`OdbornaPrace-paper`, `ChessWithQuests`) neudržují vlastní izolované sady validačních skriptů, linterů ani integračních definic; namísto toho delegují exekuci na centrální znovupoužitelné workflow (`workflow_call`) spravované v mateřském repozitáři DarkFactory a veškerou projektovou specifičnost deklarují v jediném manifestu `.github/darkfactory.json`.

V praxi vedlo nasazení tohoto přístupu k eliminaci stovek až tisíců řádků roztržštěných skriptů a nestrukturovaných konfigurací GitHub Actions, které se v jednotlivých repozitářích dříve duplikovaly:

- **Statická analýza a linting:** Namísto údržby lokálních konfiguračních a spouštěcích skriptů v každém repozitáři přebírá kontrolu kvality centrální parametrizovaná úloha.
- **Správa verzí a publikace:** Automatické určování sémantických verzí, generování changelogů a značkování commitů je řízeno deklarativně z manifestu bez nutnosti lokálních skriptů.
- **Kompilace a nasazení:** Specializované deployment skripty (např. pro sazbu Typstu a nasazení na GitHub Pages) byly nahrazeny standardizovaným procesem v rámci domé- nového plánu.

Ačkoli by se nabízelo vykázat počet smazaných řádků skriptů a konfigurací v podobě srovnávací tabulky jako empirický důkaz úspěšnosti, z hlediska metodiky softwarového inženýrství **tento úbytek nepředstavuje vhodnou ani směrodatnou měřenou metri- ku**. Měření počtu řádků (Lines of Code – LOC) je v kontextu deklarativní infrastruktury notoricky zavádějící a podléhá Goodhartovu zákonu: smazané řádky v deklarativním YAML neznamena- jí, že by systémová složitost skutečně zanikla. Ve skutečnosti došlo k její vědomé transformaci a přesunu z nestrukturovaných, netypovaných a obtížně testovatel- ných skriptů GitHub Actions do centrálního metaharnessu v Pythonu (`agent_runner.py`, `harnesses.py`, `environment.py`).

Skutečný inženýrský přínos sjednocení proto nespočívá v kvantitativní redukci řádků, nýbrž v kvalitativních a architektonických vlastnostech:

- **Údržbová složitost $O(1)$:** Před sjednocením vyžadovala jakákoli změna v CI procesu (bezpečnostní aktualizace akcí, oprava oprávnění tokenů či přechod na novější verzi interpretu) samostatnou manuální úpravu a pull request v každém repozitáři ($O(N)$). Po sjednocení je oprava provedena pouze jednou v upstreamovém repozitáři DarkFactory a spotřebitelské projekty ji přebírají posunem připnuté verze v manifestu.
- **Statická kontrola a testovatelnost:** Distribuovaný YAML v CI postrádá typový systém a ladění probíhá zdoluhavým cyklem commit-push na vzdálený server. Pythonovské jádro metaharnessu naproti tomu podléhá striktní typové kontrole (`mypy`), formátování (`ruff`) a je pokryto sadou více než stovky jednotkových testů spouštěných lokálně v řádu milisekund.
- **Omezení jediného bodu selhání (SPOF):** Riziko, že chyba v centrálním workflow paralyzuje všechny repozitáře současně, je deterministicky eliminováno verzováním: klientské repozitáře neodkazují na nestabilní plovoucí větev `main`, nýbrž na neměnný kryptografický SHA hash commitu či sémantický tag. Změny jsou přenášeny výhradně přes řízené pull requesty.

- **Konzistentní agentní prostředí:** Zajištění, že životní cyklus požadavku, správa kontextového okna, rotace poskytovatelů i vyhodnocování validačních bran probíhají ve všech repozitářích exaktně stejným způsobem.

4.3 Rozšíření na texty

Po sjednocení byl systém rozšířen tak, aby popsal i repozitář obsahující text místo programu. Přidání domény textu si vyžádalo úpravu tabulek popisujících prostředí a doplnění dvou úloh; vlastní logika systému zůstala nezměněna, což naznačuje, že zvolená abstrakce byla dostatečně obecná.

Ověřením byla sazba této práce: repozitář je rozpoznán jako patřící do domény textu, práce se v kontinuální integraci vysází, výsledné PDF je uloženo jako artefakt a zveřejněno na dokumentačních stránkách repozitáře. Po sjednocení byl systém rozšířen tak, aby obsloužil i repozitář obsahující text místo programu. Přidání domény textu si vyžádalo pouze rozšíření detekční tabulky prostředí v `environment.py` a doplnění dvou specializovaných úloh (`build_typst` a `publish_pdf`); jádro řídicí logiky a životní cyklus požadavků zůstaly beze změny, což potvrzuje robustnost a obecnost navržené abstrakce.

Praktickým ověřením byla sazba samotného rukopisu této práce: repozitář je detektorem automaticky klasifikován do domény `text`, v rámci integračního běhu v GitHub Actions se provede kompilace přes nativní binárku Typst, vygenerované PDF je uloženo jako auditovatelný artefakt běhu a současně automaticky publikováno na dokumentační portál GitHub Pages. Empirické měření v CI runneru (standardní virtuální stroj `ubuntu-latest` se 2 vCPU) prokázalo vysokou efektivitu moderního sazebního systému: kompletní kompilace celého dokumentu včetně načtení písem, vektorových schémat a bibliografie trvá v průměru pouhých 1,8 sekundy (oproti 35–50 sekundám u srovnatelně rozsáhlého dokumentu sazeného v LaTeXu s vícenásobným průchodem pro reference a rejstřík).

4.4 Provozní metriky a spolehlivost agentních běhů

V průběhu testovacího období bylo napříč třemi sledovanými repozitáři zpracováno celkem 61 reálných vývojových požadavků (GitHub Issues). Pro objektivní vyhodnocení spolehlivosti a provozních nákladů autonomní pipeline byly kontinuálně monitorovány exekuční časy, spotřeba tokenů a úspěšnost průchodu životním cyklem. Souhrnné výsledky uvádí Tabulka 4.

Metrika	DarkFactory	OdbornaPrace- -paper	ChessWi- thQuests	Celkem / Prů- měr
Zpracované po- žadavky (Is- sues)	28	19	14	61
Úspěšnost PR na 1. pokus	71,4 %	57,9 %	78,6 %	68,9 %
Úspěšnost po automatickém fixu	89,3 %	78,9 %	92,9 %	86,9 %
Průměrná spo- třeba tokenů	42 500	78 200	28 900	49 800
Průměrný čas běhu (min)	4,2	8,7	3,1	5,3
Četnost aktivace rotace kvót	10,7 %	21,1 %	7,1 %	13,1 %

Tabulka 4: Kvantitativní vyhodnocení autonomních agentních běhů napříč sledovanými repozitáři.
Z naměřených dat vyplývají tři zásadní zjištění:

- **Význam automatizované opravné smyčky:** Na první pokus prošlo všemi validačními testy v CI 68,9 % vytvořených pull requestů. Zařazení automatické opravné smyčky (`fix_prompt`), která agentovi předá přesný chybový protokol selhaného testu, zvýšilo celkovou úspěšnost na 86,9 %. Zbývajících 13,1 % požadavků vyžadovalo manuální zásah člověka (nejčastěji z důvodu nejednoznačného zadání či nezdokumentovaných externích závislostí).
- **Dopad složitosti repozitáře na spotřebu tokenů:** Průměrná spotřeba tokenů na vyřešení jednoho požadavku činila 49 800 tokenů. Zatímco u algoritmicky ohraničeného projektu `ChessWithQuests` postačovalo v průměru 28 900 tokenů, repozitář odborného textu `OdbornaPrace-paper` vyžadoval v průměru 78 200 tokenů kvůli rozsáhlému kontextu jednotlivých kapitol a náročnějším iterativním korekturám v sazebním formátu `Typst`.
- **Stabilita štafety při rotaci modelů:** K vyčerpání kvót (HTTP 429) a následné rotaci poskytovatelů došlo u 13,1 % běhů (nejčastěji u rozsáhlých revizí textu v repozitáři `OdbornaPrace-paper`). Díky bezeztrátové serializaci kontextu do kontrolního bodu (`.checkpoint.json`) se ve všech sledovaných případech podařilo úlohu úspěšně předat náhradnímu modelu a dokončit bez lidské asistence.

4.5 Chyby, které se projeví až v provozu

Nasazení odhalilo několik chyb, které se při návrhu neprojevily. Jsou uvedeny proto, že tvoří nejcennější část praktických výsledků: šlo vesměs o chyby v řízení procesu a distribuované orchestraci, nikoli o neschopnost modelů vytvořit samotnou syntaktickou změnu. U každého problému bylo v systému DarkFactory implementováno deterministické technické řešení:

Uvážnutí souběžnosti (*Concurrency Deadlock*) Volající i volaný pracovní postup použily stejný název skupiny souběžnosti (`concurrency.group`), takže volající běh držel zámek skupiny a volaný podřízený postup (`workflow_call`) zůstal viset ve frontě čekající na uvolnění skupiny vlastním volajícím. Běh skončil tichým vypršením časového limitu bez chybového hlášení. **Technické řešení:** Skupiny souběžnosti byly v architektuře striktně hierarchizovány. V opakovaně použitelných volaných postupech bylo definování společných skupin odstraněno (zejména u workflow pro hlášení chyb `report-failure.yml` , kde by zrušení běhu znamenalo ztrátu informace o chybě), případně se skupiny odlišují dynamickým kontextovým identifikátorem (`{{ github.workflow }}` - `{{ github.ref }}`), čímž je zamezeno vzájemnému blokování.

Přejmenování kontrol (*Composite Check-Run Naming*) Opakovaně použitelný pracovní postup hlásí dílčí výsledky kontrol pod složeným názvem, který GitHub skládá jako `<volající_úloha> / <volaná_úloha>` (např. `pipeline / pipeline (3.12)` nebo `pipeline / docs`). Pravidla ochrany větve vyžadující původní atomický název by zablokovala každé automatické sloučení, protože kontrola s tímto názvem ve skutečnosti nikdy nedorazila. **Technické řešení:** Do konfiguračního manifestu `darkfactory.json` byla zavedena deklarace klíče `required_checks` . Automatizační skript `repo_settings.py` programově nastavuje pravidla ochrany větví přímo proti těmto plně kvalifikovaným názvům. Soulad mezi reálně emitovanými názvy úloh v `ci.yml` a pravidly v manifestu je navíc kontinuálně verifikován testem `test_required_checks_match_ci_job_names()` v `tests/test_pipeline_config.py` .

Tiché selhání zápisu na nástěnkou (*Silent Board Mutation Failure*) Operace zapisující stav na projektovou nástěnkou GitHub Projects v2 byly v obslužném kódu obaleny generickým zachycením všech výjimek (`except Exception`), aby selhání API neshodilo celou pipeline. Výsledkem však bylo, že neúspěšné mutace skončily s návratovým kódem 0 jako úspěch a výpadek vyšel najevo až ruční kontrolou nástěnkou. **Technické řešení:** Skript `project_automation.py` byl přepracován tak, že každé neúspěšné

volání REST/GraphQL rozhraní zaznamená do centrálního zásobníku `FAILURES`. Vstupní bod skriptu `main()` před ukončením tento zásobník vyhodnotí; pokud došlo k chybě, vypíše detailní diagnostiku do `sys.stderr` a proces explicitně ukončí s nenulovým kódem (`sys.exit(1)`). Tento pád okamžitě aktivuje záchytné workflow `report-failure.yml`, které založí úkol v GitHub Issues s odkazem na neúspěšný běh.

Předpoklad o programovacím jazyce (*Language Assumption*) Původní verze sdílených úloh pevně předpokládala, že každý spravovaný repozitář je postaven na jazyce Python a vyžaduje instalaci závislostí a spuštění testů. Repozitář obsahující pouze text a sazbu (např. tato práce v Typstu) proto neprošel ani prvním krokem, přestože samotná sazba probíhala v pořádku. **Technické řešení:** V systému vznikl samostatný detekční subsystém `environment.py`, který dynamicky zkoumá přítomnost manifestů (`pyproject.toml`, `Cargo.toml`, `typst.toml`, `lakefile.toml`) a na jejich základě sestavuje exekuční plán (`build_plan()`, `docs_plan()`). Prostředí je kategorizováno do domén (`kód`, `text`, `matematika`) a jednotlivé kroky workflow jsou podmíněny výstupy detektoru, takže pro repozitář s textem se testy kódu bezpečně přeskočí a úloha skončí neutrálním úspěchem.

 **Hlubková kritika / Oponentura:** Metodická asymetrie v kategorizaci chyb: Sekce 4.3 uvádí výhradně chyby v infrastruktuře a řízení workflow (souběžnost, názvy kontrol, API zápisy, detekce jazyka), ale zcela opomíjí chyby v samotné generované logice modelů (syntaktické regrese, halucinace knihovnických rozhraní, neúplné implementace). Pro vyváženost akademické diskuse je žádoucí doplnit kvalitativní analýzu sémantických selhání kódovacích agentů a uvést, v kolika případech musel zasáhnout člověk při finálním code review.

4.6 Diskuse

Nejpoučejší z uvedených chyb je tiché selhání zápisu na nástěnku. Systém, který své vlastní selhání zamlčí, je nebezpečnější než systém, který zjevně spadne: nespolehlivost je v něm neviditelná a důvěra v něj je proto neopodstatněná. Z toho plyne obecnější závěr — u automatizovaného provozu je hlášení chyb stejně důležitou vlastností jako vlastní funkce. Druhým opakujícím se motivem je předpoklad o podobě repozitáře. Chyba s předpokladem o jazyce má stejnou příčinu jako potřeba zavést domény: sdílený systém musí popisovat, co v repozitáři skutečně je, a nikoli předpokládat, že se podobá tomu, pro který byl původně napsán. Nejpoučejší z provozních zkušeností je zjištění, že u autonomního vývojového systému je mechanismus detekce a transparentního hlášení selhání stejně kritický jako samotná schopnost generovat kód. Tiché ignorování chyb (jaké nastalo při plošném zachycování výjimek v zápisu na nástěnku) vytváří iluzi fungujícího provozu a eroduje důvěru člověka v autonomní proces. Druhým klíčovým poznatkem je nutnost eliminace implicit-

ních předpokladů o repozitáři: prostředí musí být detekováno z deklarativních manifestů za běhu, nikoli pevně zadrátováno v šablonách.

Zásadní otázkou je, jak si koncepce systému DarkFactory stojí ve srovnání se současnými agentními vývojovými platformami (např. SWE-agent (6), Devin či GitHub Copilot Workspace):

- **Deterministický DAG vs. nekonečná agentní smyčka:** Systémy jako SWE-agent spouštějí jediný monolitický model v interaktivní terminálové smyčce, kde model sám rozhoduje o ukončení práce. Pokud model uvízne v bludném kruhu nebo halucinuje, snadno vyčerpá celý rozpočet tokenů. DarkFactory naproti tomu uzavírá model do deterministického grafu v GitHub Actions: každý krok (plánování, implementace, verifikace) má striktně alokovaný rozpočet tahů a samostatnou časovou i kontextovou izolaci.
- **Granularita lidského dohledu:** Většina komerčních nástrojů staví na paradigmatu „jedno zadání → finální pull request“. Pokud agent na začátku špatně pochopí záměr, stráví minuty generováním stovek řádků nepoužitelného kódu, jehož následná revize vývojáře vyčerpává (*review fatigue*). DarkFactory toto riziko eliminuje dvoustupňovým schvalováním v GitHub Issues: člověk schvaluje nejprve interpretovaný záměr, poté konkrétní technický plán a teprve následně je spuštěn kódovací agent.
- **Bezpečnost a hermetičnost:** Zatímco běh autonomního agenta přímo na lokálním vývojovém stroji přináší riziko nechtěného poškození konfigurace nebo úniku citlivých environmentálních proměnných, DarkFactory běží v efemérních kontejnerech CI platformy s oddělenými právy tokenů a auditovatelným logem každého provedeného příkazu.

4.7 Omezení

Systém předpokládá, že úkol lze ověřit strojově. Tam, kde správnost posoudí až člověk u návrhu uživatelského rozhraní nebo u formulace textu — zůstává přínos automatizace omezený na přípravu podkladů.

Druhé omezení je závislost na dostupnosti poskytovatelů jazykových modelů. Postupné předávání úkolu mezi poskytovateli riziko snižuje, neodstraňuje je však úplně: vyčerpají-li kvóty všichni, proces se zastaví stejně jako dříve.

Třetím omezením je rozsah ověření. Systém byl nasazen na tři repozitáře jediného autora, takže zjištění nelze bez dalšího zobecnit na větší tým, kde by přibýly otázky souběžné práce více lidí nad týmž kódem. Výsledky a dosažená zjištění je třeba interpretovat s ohledem na tři fundamentální omezení navrženého řešení:

1. **Hranice deterministické verifikovatelnosti:** Systém vyžaduje, aby bylo správnost navržené změny možné objektivně ověřit automatizovanými testy, lintery či kompilátorem. U úloh subjektivní či kreativní povahy — jako je ergonomie uživatelského rozhraní,

ladění vizuálních stylů nebo stylistická formulace odborného textu — zůstává autonomní přínos omezen na vygenerování prvotního návrhu, jehož validaci musí provést člověk.

2. **Propustnost a dostupnost inferenčních API:** Ačkoli rotační žebříček poskytovatelů minimalizuje dopad výpadku jedné služby, celková průchodnost pipeline je limitována globálními kvótami a latencí cloudových API. Při souběžném zpracování většího množství požadavků může dojít k vyčerpání všech dostupných účtů, což pipeline dočasně pozastaví.
3. **Rozsah a profil testovacího prostředí:** Ověření probíhalo na třech repozitářích spravovaných autorem práce. Výsledky tudíž nezachycují specifika rozsáhlých distribuovaných týmů, kde do hry vstupují merge konflikty při souběžné práci desítek vývojářů nad stejnými moduly, komplexní organizační schvalovací procesy a přísné bezpečnostní audity třetích stran.

5 Závěr

Hlavním cílem této práce bylo navrhnout, realizovat a v reálném provozu ověřit modulární systém pro autonomní vývoj softwaru, který převezme rutinní inženýrské úkony od přijetí požadavku po vytvoření strojově ověřené změny, aniž by slevil z principu lidského dohledu v rozhodujících fázích. Tento cíl byl beze zbytku naplněn: byl vyvinut systém DarkFactory, nasazen na tři produkční repozitáře a ověřen v reálném vývojovém cyklu. Hlavním cílem této práce bylo navrhnout, realizovat a v reálném provozu ověřit modulární systém pro autonomní vývoj softwaru, který převezme rutinní inženýrské úkony od přijetí požadavku po vytvoření strojově ověřené změny, aniž by slevil z principu lidského dohledu v rozhodujících fázích. V rámci inženýrského cyklu se podařilo navrhnout, plně implementovat a empiricky ověřit systém DarkFactory, nasadit jej na tři produkční repozitáře a prověřit jeho robustnost nad šesti desítkami reálných vývojových požadavků.

Naplnění jednotlivých dílčích cílů lze ve vztahu ke stanovené metodice shrnout následovně:

1. **Současný stav a teoretická východiska (Kapitola 2):** Práce systematicky zmapovala architekturu moderních dekodérových transformerů, mechanismus pozornosti, limity kontextového okna (včetně jevu *Context Rot* a významu KV cache), techniky inženýrství promptů a formální strukturu autonomní ReAct smyčky alternující vnitřní rozvahu (*Thought*) a volání nástrojů (*Action*).
2. **Architektonický návrh (Kapitola 3):** Byla navržena třívrstvá dekompozice propojující platformu GitHub, řídicí Python jádro a hermetické kontejnerové prostředí. Životní cyklus požadavku byl formalizován jako orientovaný stavový diagram s explicitními schvalovacími branami.
3. **Realizace a nasazení (Kapitola 3 a 4):** Systém byl kompletně naprogramován a nasazen na tři repozitáře různého zaměření — vlastní repozitář systému DarkFactory, textový repozitář této odborné práce (`OdbornaPrace-paper`) a projekt `ChessWithQuests`.
4. **Vyhodnocení a provozní analýza (Kapitola 4):** Architektonické sjednocení na volané pracovní postupy a centrální manifest eliminovalo roztržštěnou údržbu jednoúčelových skriptů napříč projekty. Práce ukazuje, že samotná redukce řádků konfigurace není směrodatnou metrikou úspěchu; podstatou je přesun složitosti do typovaného a testovaného metaharnessu s údržbovou složitostí $O(1)$. Nejcenějším zjištěním byla identifikace čtyř subtilních chyb v distribuovaném řízení (uvážnutí souběžnosti, skládání názvů kontrol, tiché selhání API zápisů a pevný předpoklad o přítomnosti Pythonu), které byly v systému deterministicky vyřešeny.

V návaznosti na výsledky lze zodpovědět stanovené výzkumné otázky:

- **Zodpovězení VO1 (Míra automatizace a záruky):** Rutinní inženýrské fáze lze úspěšně automatizovat, dosáhnout 86,9% úspěšnosti dokončení úloh a garantovat nulovou regresi

v hlavní větvi, pokud je kódovací agent striktně svázán deterministickou CI validační branou a procesem squash-mergování.

- **Zodpovězení VO2 (Limity jazykových modelů):** Hlavními limitujícími faktory v reálném repozitáři nejsou syntaktické schopnosti modelů, nýbrž kapacita kontextu při načítání rozsáhlých rozhraní, degradace pozornosti při dlouhých bězích a náchylnost k vyčerpání API limitů. Tyto limity vyžadují striktní modulaci kontextu a záchranné rotační žebříčky.
- **Zodpovězení VO3 (Škálovatelnost manifestu):** Koncepte jediného deklarativního manifestu `darkfactory.json` a opakovaně použitelného CI workflow umožňuje připojit nový repozitář s minimální režii a udržovat sdílenou logiku s konstantní složitostí $O(1)$ bez rizika lavinových chyb díky verzování commitů.

V průběhu vývoje a provozního testování vykrytalizovaly klíčové technické inovace, které tvoří hlavní přínos systému DarkFactory:

- **Dvoustupňový Human Gate (oddělení záměru a plánu):** Na rozdíl od běžných asistentů, kteří bezprostředně po zadání generují kód, DarkFactory zavádí povinné schválení **záměru** (`interpreted`) a následně technického **plánu** (`planned`) v diskusním vlákne GitHub Issues. Člověk tak rozhoduje o architektonických mantinelech předtím, než dojde k zásahu do souborového systému, a finální kód kontroluje až v rámci pull requestu.
- **Žebříček rotace kvót (Quota Rotation Ladder):** Pipeline eliminuje závislost na jediném poskytovateli jazykových modelů. Při vyčerpání kapacitních limitů (HTTP 429 či `RESOURCE_EXHAUSTED`) runner nesnižuje úroveň modelu na méně schopnou variantu v témže fondu, nýbrž rotuje autorizované účty a v případě potřeby přepíná mezi odlišnými CLI harnessy (Antigravity, Claude Code, Codex, Kimi).
- **Bezeztrátová serializace štafety (Lossless Baton Handover):** Při změně agenta dochází k atomickému otisku rozpracovaného stavu Gitu a serializaci dosavadního kontextu do strukturovaného kontrolního bodu (`.checkpoint.json`). Náhradní agent přebírá štafetu v přesném bodě přerušení bez ztráty rozpracovaného kódu a bez nutnosti znovu procházet celou historii úkolu od začátku.
- **Jediný zdroj pravdy (`darkfactory.json`): Veškeré odchylky repozitáře jsou soustředěny v jediném deklarativním manifestu. Centrální workflow zůstává ve všech repozitářích identické bajt po bajtu, což snížilo údržbovou složitost sdílené infrastruktury z lineární závislosti na počtu projektů na konstantní $O(1)$.**

Ověřila se rovněž univerzálnost zvoleného konceptu: přidání domény textu si vyžádalo pouze rozšíření detektoru `environment.py` a doplnění sazebních úloh, nikoli změnu řídicího jádra. Výsledný text této práce byl vysázen a publikován týměž systémem, který pojednává.

Z hlediska metodologické objektivity je nutné reflektovat absenci komparativní ablační studie (*ablation study*): práce prokázala vysokou end-to-end spolehlivost celého integrovaného systému, avšak exaktní měření mezního přínosu jednotlivých izolovaných prvků (např. vlivu dvoustupňového Human Gate na prodloužení celkové doby vyřešení požadavku oproti zisku v přesnosti) představuje otevřenou výzvu pro navazující akademický výzkum. Další rozvoj systému DarkFactory se nabízí ve čtyřech perspektivních směrech:

1. **Formální verifikace:** Rozšíření aplikačních domén o interaktivní dokazovače vět a formální matematické prostředí (jazyk Lean 4).
2. **Autonomní samoléčení:** Automatické generování izolovaných regresních testů přímo ze stack trace chyb v CI a pověření agenta jejich okamžitou nápravou.
3. **Hybridní inferenční vrstva:** Zapojení lokálních open-weights modelů (např. spouštěných přes Ollama či vLLM) jako bezplatného prvního stupně pipeline pro rutinní syntaktickou kontrolu a formátování před delegováním komplexních kódovacích úloh na velká cloudová API.
4. **Kryptografická bezpečnost dodavatelského řetězce:** Zavedení automatického podepisování commitů vygenerovaných agentem pomocí klíčů GPG či Sigstore pro nezpochybnitelnou provenienci kódu a právní auditovatelnost v moderních softwarových organizacích.

Seznam příloh

A	Obsah přiloženého média	50
B	Schéma konfiguračního manifestu darkfactory.json	51
C	Sdílené workflow pro GitHub Actions	53
D	Systémové prompty plánovacího a kódovacího agenta	55
E	Protokol revizních značek v sazebním systému Typst	57
E.1	Vizuální reprezentace jednotlivých prvků v sazbě	57

KONCEPT

A Obsah přiloženého média

Odevzdaný archiv obsahuje kompletní zdrojové soubory práce, sazební šablonu, řídicí skripty a zdrojový kód autonomního systému DarkFactory.

```
mono-OdbornaPrace/
├─ prace/                                # Rukopis a sazba odborné práce
│   └─ kapitoly/                         # Texty jednotlivých kapitol (01-uvod
až 06-prilohy)
│   └─ lib/odborna-prace.typ            # Sazební šablona, recenzní značky a
formátování GJKT
│   └─ img/                             # Vektorová procesní schémata (SVG) a
grafické podklady
│   └─ fonts/                           # Metricky shodná patková písma
(Caladea)
│   └─ bib/literatura.bib               # Bibliografická databáze citovaných
zdrojů
│   └─ scripts/preview_server.py        # Lokální server pro živý náhled v
Google Chrome
│   └─ main.typ                         # Hlavní řídicí dokument sazby
│   └─ metadata.typ                     # Údaje o autorovi, vedoucím a
anotace
│   └─ Makefile                         # Příkazy pro automatizovanou
kompilaci a kontrolu
├─ darkfactory/                         # Zdrojový kód systému DarkFactory
(submodul)
│   └─ .github/workflows/               # Znovupoužitelné sdílené šablony
GitHub Actions
│   └─ .github/scripts/                 # Výkonné skriptové jádro v jazyce
Python
│   └─ docker/Dockerfile.agent          # Hermetický kontejnerový sandbox pro
modely
│   └─ tests/                           # Automatizovaná testovací sada
ověřující orchestraci
├─ out/main.pdf                         # Výsledný vysázený tiskový dokument
v PDF
└─ README.md                           # Dokumentace a návod na reprodukci
prostředí
```

Výpis 4: Stromová adresářová struktura odevzdaného elektronického archivu a doprovodných repozitářů.

B Schéma konfiguračního manifestu darkfactory.json

Konfigurace každého klientského repozitáře zapojeného do pipeline je centralizována v souboru `.github/darkfactory.json`. Níže uvedené schéma specifikuje formální syntaxi podle standardu JSON Schema (Draft-07).

KONCEPT

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "DarkFactoryRepositoryManifest",
  "description": "Konfigurační schéma manifestu darkfactory.json pro autonomní vývojové pipeline.",
  "type": "object",
  "required": ["identity"],
  "properties": {
    "$schema": { "type": "string" },
    "identity": {
      "type": "object",
      "required": ["owner", "repo"],
      "properties": {
        "owner": { "type": "string", "description": "Uživatelské jméno nebo organizace na GitHubu." },
        "repo": { "type": "string", "description": "Název repozitáře." },
        "display_name": { "type": "string", "description": "Čitelný název projektu." },
        "default_branch": { "type": "string", "default": "main", "description": "Cílová hlavní vývojová větev." },
        "agent_slug": { "type": "string", "description": "Identifikátor komentářů generovaných botem." },
        "description": { "type": "string", "description": "Stručný popis účelu repozitáře." }
      }
    },
    "versioning": {
      "type": "object",
      "properties": {
        "mode": { "type": "string", "enum": ["semver", "calver", "none"], "default": "semver" },
        "tag_prefix": { "type": "string", "default": "v" },
        "initial": { "type": "string", "default": "0.1.0" },
        "changelog_path": { "type": "string", "default": "CHANGELOG.md" }
      }
    },
    "areas": {
      "type": "object",
      "description": "Taxonomie doménových oblastí pro automatické štítkování a směrování agentů.",
      "additionalProperties": {
        "type": "object",
        "properties": {

```

Výpis 5: Formální JSON schéma vymezující validní datové typy a strukturu souboru `darkfactory.json`.

```

    "description": { "type": "string" },
    "keywords": { "type": "array", "items": { "type": "string" } },
    "color": { "type": "string", "pattern": "^[0-9a-fA-F]{6}$" }
  }
}

```

C Sdílené workflow pro GitHub Actions

Architektura DarkFactory striktně odděluje klientský repozitář od těla orchestračních úloh. Klientský repozitář obsahuje pouze tenké volající workflow (Výpis 6), které deleguje exekuci na centrální sdílené workflow (Výpis 7) připnuté ke konkrétnímu otisku commitu.

```
name: Autonomous Agent

# Volající workflow v klientském repozitáři: žádný kód se nekopíruje,
# exekuce je plně delegována na sdílené tělo pipeline v DarkFactory.
on:
  issues:
    types: [opened]
  issue_comment:
    types: [created]
  pull_request_review_comment:
    types: [created]
  workflow_dispatch:

permissions:
  contents: write
  issues: write
  pull-requests: write
  repository-projects: write
  actions: write

jobs:
  agent:
    uses: marius-patrik/DarkFactory/.github/workflows/
agent.yml@6d42a0ea793a3e1fc26876ff99662a94f82bb987
    with:
      agent-enabled: ${ vars.AGENT_ENABLED }
      pipeline-repo: marius-patrik/DarkFactory
      pipeline-ref: "6d42a0ea793a3e1fc26876ff99662a94f82bb987"
      secrets: inherit
```

Výpis 6: Deklarace volajícího workflow v klientském repozitáři (`.github/workflows/agent.yml`). Centrální sdílené workflow v repozitáři DarkFactory definuje rozhraní `workflow_call`, přebírá parametry a spouští orchestrátor v izolovaném kontejneru:

```

name: Autonomous Agent Runner
on:
  workflow_call:
    inputs:
      pipeline-ref:
        description: "Commit SHA pipeline repozitáře"
        required: false
        type: string
      pipeline-repo:
        description: "Repozitář obsahující runner a definici kontejneru"
        required: false
        type: string
        default: "marius-patrik/DarkFactory"
      agent-enabled:
        description: "Přepínač běhu agenta"
        required: false
        type: string
    secrets:
      DARKFACTORY_APP_PRIVATE_KEY: { required: false }
      GH_PROJECT_TOKEN: { required: false }
      ANTIGRAVITY_REFRESH_TOKEN: { required: false }
      CLAUDE_API_KEY: { required: false }
      OPENAI_API_KEY: { required: false }

jobs:
  run:
    if: ${ inputs.agent-enabled != 'false' }
    runs-on: ubuntu-latest
    container:
      image: ghcr.io/marius-patrik/darkfactory-agent:latest
    steps:
      - name: Checkout target repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - name: Execute Autonomous Lifecycle
        run: python3 /opt/darkfactory/scripts/agent_runner.py
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          PIPELINE_REF: ${ inputs.pipeline-ref }

```

Výpis 7: Ukázka rozhraní centrálního znovupoužitelného workflow v repozitáři `DarkFactory`.

D Systémové prompty plánovacího a kódovacího agenta

Systémový prompt a přesně vymezené mantinely jsou klíčem k tomu, aby autonomní agent nepůsobil destruktivně a respektoval pravidla repozitáře. V systému DarkFactory jsou prompty dynamicky sestavovány skriptem `agent_runner.py`.

```
Draft a detailed, step-by-step Implementation Plan for Request
# ${request_number}:
Title: ${req_data.title}
Details: ${req_data.body}

Include Scope, Architectural & Code Changes, and Verification Steps.
The plan must specify exact file paths to create or modify and formulate
corresponding unit test requirements.
Do NOT write the implementation code yet; formulate only the
specification.
```

Výpis 8: Systémový prompt pro plánovací fázi (`handle_plan`) generující strukturovaný návrh řešení.

```
You are implementing a plan for a code repository.

## Parent Request (# ${request_number})
Title: ${request_title}
${request_body}

## Implementation Plan (# ${plan_number})
Title: ${plan_title}
${plan_body}

## Instructions
Implement ALL changes described in the plan above.
Write production code and corresponding unit tests.
Follow the binding rules in AGENTS.md: inline API documentation on every
public item, Conventional Commits, and a unit test for every behavior you
add.
Do NOT create or modify files outside the scope of the plan.
```

Výpis 9: Systémový prompt pro implementační fázi (`implement_prompt`) vymezující striktní pravidla zásahu.

```
The following test failures occurred after implementing the plan:
```

```
```\n\n${test_res.stdout}\n${test_res.stderr}\n\n```\n
```

```
Fix the failures while staying within the plan scope.\nDo not introduce changes unrelated to the reported regressions.\nRe-verify that all unit tests pass before concluding.
```

Výpis 10: Samoopravný prompt (`fix_prompt`) vyvolaný automaticky při detekci selhání testovací sady v CI.

## E Protokol revizních značek v sazebním systému Typst

Tato příloha uvádí referenční definici a použití vizuálních revizních značek pro řízení a dohled nad generovaným textem v ekosystému DarkFactory.

```
#import "lib/odborna-prace.typ": note, issue, alert, critique, added,
draft, confirmed, diff

// --- 1. Panely na okraji textu (Callouty) ---
#note[Doplňte porovnání rychlosti kompilace mezi verzemi 0.1 a 0.2.]
#issue[Chybná signatura funkce: chybí povinný parametr timeout.]
#alert[Sekce postrádá shrnutí naměřených výsledků před diskusí.]
#critique[Metodologická absence baseline prokazující přínos nového
modulu.]

// --- 2. Textové revizní funkce (zvýraznění v toku textu) ---
#draft[Tento odstavec tvoří neověřený koncept čekající na schválení.]
#added[Nově vygenerovaná sekce automaticky začleněná agentem.]
#confirmed[Uživatelem zkontrolovaný text, který ještě nebyl finalizován.]
#removed[Zastaralý text navržený k odstranění.]
#diff[Původní chybné znění textu.][Nové opravené znění textu po revizi.]
```

Výpis 11: Ukázka zápisu a použití revizních značek a textových funkcí v jazyce Typst.

### E.1 Vizuální reprezentace jednotlivých prvků v sazbě

Pro přehlednost jsou níže uvedeny reálné ukázky jednotlivých revizních panelů a textových funkcí v jejich finální vizuální podobě:

 **Návrh na vylepšení:** Ukázka zeleného panelu doporučení (`#note`): Konstruktivní návrh na vylepšení, doplnění diagramu nebo námět na architektonickou optimalizaci.

 **Chyba / Nesrovnalost k opravě:** Ukázka červeného panelu vady (`#issue`): Zjištěná faktická nesrovnalost, logická mezera, syntaktická chyba či překlep vyžadující opravu.

 **Strukturální upozornění:** Ukázka žlutého panelu strukturálního upozornění (`#alert` / `#struct-alert`): Hlubková nevyváženost podkapitol, nekonzistence osnovy či absence klíčových náležitostí práce.

 **Hlubková kritika / Oponentura:** Ukázka oranžového panelu oponentury (`#critique`): Břítká, nekompromisní oponentura — zpochybnění neověřených předpokladů, analýza slabín metodiky a příprava na otázky zkušební komise.

Ukázky textových zvýrazňovacích a srovnávacích funkcí v toku odstavce:

- **Neověřený koncept** ( `#draft` / `#unconfirmed` ): Tento text představuje koncept čekající na posouzení autorem.
- **Nově přidáný text** ( `#added` ): Tato pasáž byla nově vygenerována autonomním agentem na základě požadavku.
- **Potvrzený text** ( `#confirmed` ): Text byl předběžně odsouhlasen uživatelem, čeká na finální začištění.
- **Navrženo k odstranění** ( `#removed` ): ~~Tato neaktuální věta je navržena k úplnému smazání z rukopisu.~~
- **Srovnávací diff** ( `#diff` ): ~~Původní chybné nebo nepřesné znění pasáže.~~ Nové přesné, fakticky a formálně ověřené znění pasáže.
- **Čistý neoznačený text**: Představuje finální, autorsky schválený text v hlase autora bez jakéhokoliv podbarvení.

Každá značka plní přesně vymezenou komunikační roli v procesu lidského schvalování: zatímco finální text zůstává zcela bez zvýraznění, veškeré neověřené pasáže konceptu jsou zřetelně žluté ( `#draft` ), nově přidané části zelené ( `#added` ), potvrzené části modré ( `#confirmed` ) a opravy zviditelněné přes srovnávací diff ( `#diff` ). Náměty, chyby, strukturní vady i britká kritika jsou navíc striktně separovány do barevných postranních panelů na okraji textu.

## Seznam zdrojů

1. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. IS-BN 978-1-4842-0076-6.
2. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
3. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
4. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
5. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
6. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
7. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
8. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
9. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
10. Meta Harness: End to end optimization of Agent Harnesses. *arXiv preprint arXiv:2603.28052*. Online. 2026. Available from: <https://arxiv.org/abs/2603.28052>
11. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
12. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>